
Randomized Algorithms for Primality Testing

A Journey from Exponential Traps to Probabilistic Certainty

CS648: Randomized Algorithms

Group ID: 231006

April 1, 2026

Abstract

We present a detailed, self-contained account of randomized primality testing, tracing the journey from the naive—and exponentially slow—trial division algorithm, through Fermat’s Little Theorem and the catastrophic failure of Carmichael numbers, to the elegant Miller–Rabin probabilistic test. All major theorems are proved from first principles without invoking abstract group theory; instead we rely on elementary modular arithmetic and explicit counting arguments. Empirical analysis with Python code confirms that the Miller–Rabin test achieves an error probability below $\frac{1}{4^k}$ after k rounds, with experimental measurements consistently beating the theoretical bound.

Contents

1	The Efficiency Illusion: Why Trial Division Fails	3
1.1	The Naive Algorithm	3
1.2	The Exponential Trap	3
1.3	Why Primality Testing Matters	4
2	Fermat's Little Theorem and the Fermat Primality Test	4
2.1	Statement and Proof	4
2.2	The Fermat Primality Test	5
2.3	Modular Exponentiation	6
3	The Coprime Structure and the 50% Witness Guarantee	6
3.1	Non-Coprime Bases Always Witness	6
3.2	The Injection Argument: Witnesses $\geq$ Liars	6
3.3	Why Carmichael Numbers Break Everything	7
4	The Miller–Rabin Test: Springing the Square-Root Trap	8
4.1	The Core Idea: Non-Trivial Square Roots of 1	8
4.2	Factoring Out Powers of Two	8
4.3	The Primality Conditions	9
4.4	Witnesses and Nonwitnesses	9
4.5	The Algorithm	10
4.6	Worked Examples	10
5	The 75% Witness Guarantee: Full Proof	11
5.1	Case 1: $n = p^\alpha$ is a Prime Power ($\alpha \geq 2$)	11
5.2	Case 2: n Has at Least Two Distinct Prime Factors	12
5.3	Sharpness of the $1/4$ Bound	14
6	Error Analysis and Probability Bounds	14
6.1	Error Probability After k Rounds	14
6.2	Choosing k in Practice	15
6.3	Deterministic Behaviour for Small Numbers	15
6.4	The GRH Connection	17
7	Empirical Analysis and Python Code	18
7.1	Liar Fraction Analysis	18
7.2	Worst-Case Composites	23
7.3	Witness Reliability across Bases	32
7.4	Empirical Error Probability Table	36

7.5	Fermat vs. Miller–Rabin Comparison	42
7.6	Discussion of Results	51
8	Complexity Analysis	51
8.1	Time Complexity of Miller–Rabin	51
8.2	Comparison Table	52
9	A Second Proof of the 75% Bound (Squarefree Case)	52
9.1	Setup	52
9.2	Counting Nonwitnesses Directly	52
9.3	Bounding the Count	53
10	Historical Notes and Connections	53
10.1	Timeline	53
10.2	Miller’s Original Test vs Rabin’s Version	54
10.3	The AKS Algorithm	54
10.4	Connection to RSA Cryptography	54
11	Summary of Proofs and Theorems	55
12	Conclusions	55

1 The Efficiency Illusion: Why Trial Division Fails

1.1 The Naive Algorithm

The most natural approach to deciding whether a positive integer n is prime is *trial division*: check whether any integer d in the range $2 \leq d \leq \sqrt{n}$ divides n . If no such divisor exists, n is prime; otherwise it is composite.

Algorithm 1 Trial Division Primality Test

Require: Integer $n \geq 2$

Ensure: PRIME or COMPOSITE

```

1: if  $n = 2$  then return PRIME
2: end if
3: if  $n$  is even then return COMPOSITE
4: end if
5: for  $d \leftarrow 3$  to  $\lfloor \sqrt{n} \rfloor$  step 2 do
6:   if  $d \mid n$  then return COMPOSITE
7:   end if
8: end for return PRIME
  
```

The loop runs at most $\frac{\sqrt{n}}{2}$ times, so the time complexity is $O(\sqrt{n})$. This looks fast—until we remember how “input size” is defined in the RAM model.

1.2 The Exponential Trap

Key Insight. In theoretical computer science, the *input size* of an integer n is its *bit-length*, not its magnitude. If we use x bits to write n in binary, then $x = \lfloor \log_2 n \rfloor + 1$, so $n \approx 2^x$. An algorithm is *efficient* only if its running time is polynomial in x .

Substituting $n = 2^x$ into the trial division complexity:

$$O(\sqrt{n}) = O(\sqrt{2^x}) = O(2^{x/2}).$$

This is *exponential* in the input size x . For a 1024-bit prime ($x = 1024$), trial division requires roughly 2^{512} operations—far beyond the reach of any classical computer for the lifetime of the universe.

Example 1.1. A modern supercomputer performs $\approx 10^{18}$ operations per second. The age of the universe is $\approx 4.3 \times 10^{17}$ seconds. So in the universe’s entire lifetime, a supercomputer completes roughly 4.3×10^{35} operations. But $2^{512} \approx 1.3 \times 10^{154}$, which is *incomprehensibly* larger. Trial division is hopeless for cryptographic-size primes.

1.3 Why Primality Testing Matters

Primality testing is not merely an academic curiosity. The RSA cryptosystem, which secures most of the internet's financial transactions, requires generating large primes (typically 1024–4096 bits). A slow primality test would make key generation impractical. We therefore need an algorithm that runs in time *polynomial* in the bit-length $x = \log_2 n$.

2 Fermat's Little Theorem and the Fermat Primality Test

2.1 Statement and Proof

Theorem 2.1 (Fermat's Little Theorem). *Let p be a prime and a an integer with $1 \leq a < p$. Then*

$$a^{p-1} \equiv 1 \pmod{p}.$$

Proof. We give a complete, elementary proof.

Step 1: Setting up the two sets. Consider the set of non-zero residues modulo p :

$$S = \{1, 2, 3, \dots, p-1\}.$$

Multiply every element of S by a to obtain:

$$S' = \{a, 2a, 3a, \dots, (p-1)a\},$$

where we regard each element as its residue modulo p .

Step 2: Showing S' is a permutation of S modulo p . We must verify two things:

- (i) *No element of S' is 0 mod p .* If $ja \equiv 0 \pmod{p}$ for some $1 \leq j \leq p-1$, then $p \mid ja$. Since p is prime and $1 \leq j \leq p-1$, we have $\gcd(j, p) = 1$. Similarly, $1 \leq a \leq p-1$ gives $\gcd(a, p) = 1$. Hence $p \nmid j$ and $p \nmid a$, so $p \nmid ja$ —contradiction.
- (ii) *All elements of S' are distinct modulo p .* Suppose $j_1 a \equiv j_2 a \pmod{p}$ for $1 \leq j_1, j_2 \leq p-1$. Then $p \mid (j_1 - j_2)a$. Since $\gcd(a, p) = 1$, we get $p \mid (j_1 - j_2)$. But $|j_1 - j_2| < p$, so $j_1 = j_2$.

Therefore S' contains $p-1$ distinct, non-zero residues modulo p , which means $S' \equiv S \pmod{p}$ as sets.

Step 3: Taking the product. Since S' is a rearrangement of S , their products modulo p are equal:

$$a \cdot 2a \cdot 3a \cdots (p-1)a \equiv 1 \cdot 2 \cdot 3 \cdots (p-1) \pmod{p}.$$

The left side factors as $a^{p-1} \cdot (p-1)!$, so

$$a^{p-1} \cdot (p-1)! \equiv (p-1)! \pmod{p}.$$

Step 4: Cancelling the factorial. Since p is prime and $1 \leq j \leq p-1$ for every factor of $(p-1)!$, we have $\gcd((p-1)!, p) = 1$. Hence $(p-1)!$ is invertible modulo p , and we may cancel it from both sides:

$$a^{p-1} \equiv 1 \pmod{p}. \quad \square$$

2.2 The Fermat Primality Test

Fermat's theorem gives us a *necessary* condition for primality: if n is prime then $a^{n-1} \equiv 1 \pmod{n}$ for all a with $\gcd(a, n) = 1$. Contrapositive: if $a^{n-1} \not\equiv 1 \pmod{n}$, then n is *definitely composite*.

Definition 2.2 (Fermat Witness and Liar). Let n be an odd composite. A base a with $1 \leq a \leq n-1$ is called:

- a **Fermat witness** for n if $a^{n-1} \not\equiv 1 \pmod{n}$ (it *proves* n is composite), and
- a **Fermat liar** for n if $a^{n-1} \equiv 1 \pmod{n}$ (it *falsely suggests* n is prime).

Algorithm 2 Fermat Primality Test (k rounds)

Require: Odd integer $n \geq 3$, number of trials $k \geq 1$

Ensure: PROBABLYPRIME or COMPOSITE

```

1: for  $i \leftarrow 1$  to  $k$  do
2:   Pick  $a$  uniformly at random from  $\{2, 3, \dots, n-2\}$ 
3:   if  $a^{n-1} \not\equiv 1 \pmod{n}$  then return COMPOSITE            $\triangleright a$  is a Fermat witness
4:   end if
5: end for return PROBABLYPRIME

```

The key operation $a^{n-1} \bmod n$ is computed via *fast modular exponentiation* (repeated squaring) in $O(\log n)$ multiplications of numbers each at most $n-1$. Each such multiplication on x -bit numbers costs $O(x^2)$ bit operations (or $O(x^{1.585})$ with Karatsuba). Thus each round runs in $O((\log n)^3)$ bit operations—genuinely polynomial in $x = \log n$.

2.3 Modular Exponentiation

Algorithm 3 Fast Modular Exponentiation: $\text{powmod}(a, e, n)$

Require: $a, e, n \in \mathbb{Z}$, $n > 0$, $e \geq 0$

Ensure: $a^e \bmod n$

```

1: result  $\leftarrow 1$ ;  $a \leftarrow a \bmod n$ 
2: while  $e > 0$  do
3:   if  $e$  is odd then
4:     result  $\leftarrow$  result  $\cdot a \bmod n$ 
5:   end if
6:    $e \leftarrow \lfloor e/2 \rfloor$ ;  $a \leftarrow a^2 \bmod n$ 
7: end while return result

```

Correctness sketch. At every step we maintain the invariant $\text{result} \cdot a^e \equiv a^{e_0} \pmod{n}$, where e_0 is the original exponent. When $e = 0$ the invariant gives $\text{result} \equiv a^{e_0} \pmod{n}$. $\square$

3 The Coprime Structure and the 50% Witness Guarantee

3.1 Non-Coprime Bases Always Witness

Lemma 3.1. *Let n be composite and a an integer with $\gcd(a, n) = d > 1$. Then $a^{n-1} \not\equiv 1 \pmod{n}$, so a is a Fermat witness.*

Proof. Suppose for contradiction that $a^{n-1} \equiv 1 \pmod{n}$. Then $n \mid a^{n-1} - 1$, so there exists an integer m with $a^{n-1} - 1 = mn$. Rearranging: $a^{n-1} - mn = 1$. Since $d \mid a$ we have $d \mid a^{n-1}$, and since $d \mid n$ we have $d \mid mn$. Therefore $d \mid (a^{n-1} - mn) = 1$, which forces $d = 1$, contradicting $d > 1$. $\square$

3.2 The Injection Argument: Witnesses $\geq$ Liars

Now we handle the harder case: bases a coprime to n .

Theorem 3.2 (50% Witness Guarantee for Non-Carmichael Composites). *Let n be an odd composite that is not a Carmichael number. Then at least half of all bases $a \in \{1, 2, \dots, n-1\}$ with $\gcd(a, n) = 1$ are Fermat witnesses for n .*

Proof. Let

$$L = \{a \in \{1, \dots, n-1\} : \gcd(a, n) = 1, a^{n-1} \equiv 1 \pmod{n}\}$$

be the set of Fermat liars. Since n is not a Carmichael number, there exists a coprime base w with $w^{n-1} \not\equiv 1 \pmod{n}$ —that is, w is a Fermat witness coprime to n .

Define the map $f : L \rightarrow \{1, \dots, n-1\}$ by

$$f(x) = wx \pmod{n}.$$

We claim every element of $f(L)$ is a Fermat witness:

$$(wx)^{n-1} = w^{n-1} \cdot x^{n-1} \equiv w^{n-1} \cdot 1 = w^{n-1} \not\equiv 1 \pmod{n}.$$

Now we show f is *injective*. If $wx_1 \equiv wx_2 \pmod{n}$ and $\gcd(w, n) = 1$, then w has a multiplicative inverse modulo n , so multiplying both sides by w^{-1} gives $x_1 \equiv x_2 \pmod{n}$. Since $1 \leq x_1, x_2 \leq n-1$, we get $x_1 = x_2$.

Therefore $|f(L)| = |L|$: every liar maps to a *distinct* witness. Since $f(L)$ and L are disjoint (a base cannot be both a witness and a liar), we have

$$(\text{number of witnesses coprime to } n) \geq |L| \geq (\text{number of liars coprime to } n).$$

Hence at most half the coprime bases are liars. □

3.3 Why Carmichael Numbers Break Everything

Definition 3.3 (Carmichael Number). An odd composite integer n is a **Carmichael number** if $a^{n-1} \equiv 1 \pmod{n}$ for *every* integer a with $\gcd(a, n) = 1$.

For a Carmichael number the set L of liars *equals* the entire set of coprime bases—so the injection argument gives 0 witnesses among the coprimes. The only witnesses are the non-coprime bases from Lemma 3.1, but:

The Needle-in-a-Haystack Problem. If $n = pq$ for two large primes (e.g. each 500 bits), then the fraction of integers in $\{1, \dots, n-1\}$ that share a factor with n is approximately

$$1 - \frac{\phi(n)}{n} = 1 - \frac{(p-1)(q-1)}{pq} = \frac{p+q-1}{pq} \approx \frac{1}{p} + \frac{1}{q} \approx 2^{-499}.$$

So a random base has probability $\approx 2^{-499}$ of being a non-coprime witness. The Fermat test is *completely useless* against cryptographic-size Carmichael numbers.

The smallest Carmichael number is $561 = 3 \times 11 \times 17$.

Example 3.4 (Verifying 561 is a Carmichael number). We use **Korselt's criterion**: n is a Carmichael number if and only if n is squarefree and $(p-1) \mid (n-1)$ for every prime

factor p of n .

For $n = 561$: $n - 1 = 560$.

$$\begin{aligned} p = 3: \quad 3 - 1 = 2 \mid 560 \checkmark \\ p = 11: \quad 11 - 1 = 10 \mid 560 \checkmark \\ p = 17: \quad 17 - 1 = 16 \mid 560 \checkmark \end{aligned}$$

All conditions hold, so 561 is a Carmichael number.

Korselt's Criterion (proof sketch). If $(p - 1) \mid (n - 1)$ for every prime $p \mid n$, then by Fermat's little theorem $a^{p-1} \equiv 1 \pmod{p}$, so $a^{n-1} = (a^{p-1})^{(n-1)/(p-1)} \equiv 1 \pmod{p}$ for each p , and the Chinese Remainder Theorem gives $a^{n-1} \equiv 1 \pmod{n}$.

4 The Miller–Rabin Test: Springing the Square-Root Trap

4.1 The Core Idea: Non-Trivial Square Roots of 1

Lemma 4.1 (Square Roots of 1 Modulo a Prime). *If p is an odd prime and $x^2 \equiv 1 \pmod{p}$, then $x \equiv 1 \pmod{p}$ or $x \equiv -1 \pmod{p}$.*

Proof. $x^2 \equiv 1 \pmod{p}$ means $p \mid x^2 - 1 = (x - 1)(x + 1)$. Since p is prime, $p \mid (x - 1)$ or $p \mid (x + 1)$, giving $x \equiv 1$ or $x \equiv -1 \pmod{p}$. $\square$

Intuition. If n is prime and we discover some x with $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv \pm 1 \pmod{n}$ —a *non-trivial* square root of 1—then by Lemma 4.1 n cannot be prime. The Miller–Rabin test is a systematic procedure for hunting such non-trivial square roots.

4.2 Factoring Out Powers of Two

Since n is odd, $n - 1$ is even. Write

$$n - 1 = 2^s \cdot d, \quad d \text{ odd}, \quad s \geq 1.$$

Starting from Fermat's relation $a^{n-1} \equiv 1 \pmod{n}$ (which holds when n is prime), we can work backwards:

$$a^{n-1} = (a^{2^{s-1}d})^2 = \left((a^{2^{s-2}d})^2 \right)^2 = \dots$$

This yields the *Miller–Rabin sequence* of $s + 1$ values:

$$a^d, a^{2d}, a^{4d}, \dots, a^{2^{s-1}d}, a^{2^s d} = a^{n-1}.$$

Each term is the square of the previous one.

4.3 The Primality Conditions

Theorem 4.2 (Miller–Rabin Necessary Condition for Primes). *If n is an odd prime and $n - 1 = 2^s d$ with d odd, then for every a with $\gcd(a, n) = 1$, the Miller–Rabin sequence satisfies at least one of:*

- (i) $a^d \equiv 1 \pmod{n}$, or
- (ii) $a^{2^i d} \equiv -1 \pmod{n}$ for some $i \in \{0, 1, \dots, s-1\}$.

Proof. By Fermat’s Little Theorem (Theorem 2.1), $a^{n-1} \equiv 1 \pmod{n}$, i.e. $a^{2^s d} \equiv 1 \pmod{n}$.

Now consider the sequence $b_0 = a^d$, $b_1 = b_0^2 = a^{2d}$, $\dots$, $b_s = a^{2^s d} \equiv 1$. The last term is 1. Walk backwards: if $b_s = 1$, consider b_{s-1} . Since $b_{s-1}^2 \equiv 1 \pmod{n}$ and n is prime, by Lemma 4.1, $b_{s-1} \equiv \pm 1 \pmod{n}$.

- If $b_{s-1} \equiv -1$: condition (ii) holds for $i = s-1$. Done.
- If $b_{s-1} \equiv 1$: walk back to b_{s-2} . Again $b_{s-2}^2 = b_{s-1} \equiv 1$, so $b_{s-2} \equiv \pm 1$.

Continuing this process, either we encounter -1 at some step (giving condition (ii)), or every $b_i = 1$, in which case $b_0 = a^d \equiv 1 \pmod{n}$ (condition (i)). $\square$

4.4 Witnesses and Nonwitnesses

Definition 4.3 (Miller–Rabin Witness and Nonwitness). For odd $n > 1$ with $n - 1 = 2^s d$, d odd, and $a \in \{1, \dots, n-1\}$:

- a is a **Miller–Rabin witness** for n if $a^d \not\equiv 1 \pmod{n}$ and $a^{2^i d} \not\equiv -1 \pmod{n}$ for all $i \in \{0, \dots, s-1\}$.
- a is a **Miller–Rabin nonwitness** (or *strong pseudoprime to base a*) if one of the two conditions in Theorem 4.2 holds.

A Miller–Rabin witness proves n is composite. An odd prime has *no* Miller–Rabin witnesses.

4.5 The Algorithm

Algorithm 4 Miller–Rabin Primality Test

Require: Odd integer $n \geq 3$, number of rounds $k \geq 1$

Ensure: PROBABLYPRIME or COMPOSITE

```

1: Write  $n - 1 = 2^s \cdot d$  with  $d$  odd ▷ Factor out powers of 2
2: for round  $\leftarrow 1$  to  $k$  do
3:   Pick  $a$  uniformly at random from  $\{2, 3, \dots, n - 2\}$ 
4:    $x \leftarrow a^d \bmod n$  ▷ Fast modular exponentiation
5:   if  $x = 1$  or  $x = n - 1$  then
6:     continue ▷ Nonwitness condition (i): passes this round
7:   end if
8:   for  $i \leftarrow 1$  to  $s - 1$  do
9:      $x \leftarrow x^2 \bmod n$ 
10:    if  $x = n - 1$  then
11:      goto next round ▷ Nonwitness condition (ii): passes
12:    end if
13:    if  $x = 1$  then return COMPOSITE ▷ Reached 1 without seeing  $-1$ : non-trivial
    sqrt found
14:    end if
15:  end for return COMPOSITE ▷ Final  $a^{n-1} \not\equiv 1 \pmod{n}$ 
16: end for return PROBABLYPRIME

```

4.6 Worked Examples

Example 4.4 ($n = 561$, $a = 2$). $n - 1 = 560 = 2^4 \cdot 35$, so $s = 4$, $d = 35$. Compute the sequence 2^{35} , 2^{70} , 2^{140} , $2^{280} \pmod{561}$:

$$\begin{aligned}
 2^{35} &\equiv 263 \pmod{561} \\
 2^{70} &= 263^2 \equiv 166 \pmod{561} \\
 2^{140} &= 166^2 \equiv 67 \pmod{561} \\
 2^{280} &= 67^2 \equiv 1 \pmod{561}
 \end{aligned}$$

The sequence is $(263, 166, 67, 1)$. The last element is 1, but the element immediately preceding it is $67 \not\equiv \pm 1 \pmod{561}$. So we have found a non-trivial square root of 1: $67^2 \equiv 1 \pmod{561}$. Miller–Rabin correctly declares 561 COMPOSITE.

Example 4.5 ($n = 29341$ (Carmichael), $a = 3$). $n - 1 = 29340 = 2^2 \cdot 7335$, so $s = 2$, $d = 7335$. The Miller–Rabin sequence for $a = 3$ is:

$$3^{7335} \equiv 22569 \pmod{29341}, \quad 3^{14670} \equiv 1 \pmod{29341}.$$

The value $22569 \not\equiv 1$ and $22569 \not\equiv 29340 = -1$, so 3 is a Miller–Rabin witness for 29341.

Even this Carmichael number is caught!

Example 4.6 ($n = 13$ (prime), $a = 5$). $n - 1 = 12 = 2^2 \cdot 3$, so $s = 2$, $d = 3$.

$$5^3 = 125 \equiv 125 - 9 \times 13 = 125 - 117 = 8 \pmod{13}, \quad 8^2 = 64 \equiv 64 - 4 \times 13 = 12 \equiv -1 \pmod{13}.$$

Condition (ii) holds at $i = 1$, so 5 is a nonwitness and the test correctly does *not* declare 13 composite.

5 The 75% Witness Guarantee: Full Proof

This section proves the central theorem that makes Miller–Rabin a practically useful algorithm. We avoid abstract group theory, instead working directly with counting arguments and modular arithmetic.

Theorem 5.1 (Rabin–Monier, 1980). *Let $n > 1$ be an odd composite. Then the fraction of bases $a \in \{2, 3, \dots, n - 2\}$ that are Miller–Rabin nonwitnesses for n is strictly less than $\frac{1}{4}$. Equivalently,*

$$\Pr_{a \sim \{2, \dots, n-2\}} [a \text{ is a nonwitness for } n] < \frac{1}{4}.$$

We split the proof into two main cases based on the structure of n .

Preliminary: Every Nonwitness Satisfies $a^{n-1} \equiv 1$

If a is a nonwitness then either $a^d \equiv 1$ or $a^{2^i d} \equiv -1$ for some i . In both cases:

$$a^{2^i d} \equiv -1 \implies a^{2^{i+1} d} \equiv 1 \implies a^{2^s d} = a^{n-1} \equiv 1 \pmod{n}.$$

So every nonwitness satisfies the Fermat equation. In particular, every nonwitness is coprime to n .

5.1 Case 1: $n = p^\alpha$ is a Prime Power ($\alpha \geq 2$)

Lemma 5.2. *If $n = p^\alpha$ with p prime, $\alpha \geq 2$, then the number of $a \in \{1, \dots, n - 1\}$ that are Miller–Rabin nonwitnesses for n is exactly $p - 1$. Their proportion is*

$$\frac{p - 1}{p^\alpha - 1} = \frac{1}{1 + p + p^2 + \dots + p^{\alpha-1}} \leq \frac{1}{1 + p} \leq \frac{1}{4}.$$

Proof. Step 1. We show that a is a nonwitness for $n = p^\alpha$ if and only if $a^{p-1} \equiv 1 \pmod{p^\alpha}$.

($\Rightarrow$) A nonwitness satisfies $a^{n-1} \equiv 1 \pmod{n}$. By Euler's theorem, $a^{\phi(n)} = a^{p^{\alpha-1}(p-1)} \equiv 1 \pmod{n}$. Now $\gcd(n-1, \phi(n)) = \gcd(p^{\alpha}-1, p^{\alpha-1}(p-1))$. Since $p^{\alpha-1}$ and $p^{\alpha}-1$ are coprime (the latter is not divisible by p), we get $\gcd(n-1, \phi(n)) = \gcd(p^{\alpha}-1, p-1) = p-1$ (because $p-1 \mid p^{\alpha}-1$ and $p-1$ is the largest common factor). Thus $a^{p-1} \equiv 1 \pmod{n}$.

($\Leftarrow$) Suppose $a^{p-1} \equiv 1 \pmod{p^{\alpha}}$. Write $p-1 = 2^f \ell$ with ℓ odd. Since $p-1 \mid p^{\alpha}-1 = n-1 = 2^s d$, we have $f \leq s$ and $\ell \mid d$. Consider $b = a^{\ell} \pmod{p^{\alpha}}$; then $b^{2^f} \equiv 1$. Let j be minimal with $b^{2^j} \equiv 1$. If $j = 0$: $a^{\ell} \equiv 1$, so $a^d \equiv 1$ (since $\ell \mid d$). If $j \geq 1$: $b^{2^{j-1}} \not\equiv 1$ but $(b^{2^{j-1}})^2 \equiv 1$.

We now show $b^{2^{j-1}} \equiv -1 \pmod{p^{\alpha}}$. We have $p^{\alpha} \mid (b^{2^{j-1}} - 1)(b^{2^{j-1}} + 1)$. The two factors differ by 2, so p (odd prime) cannot divide both. Hence p^{α} divides one of them. Since $b^{2^{j-1}} \not\equiv 1$, we get $b^{2^{j-1}} \equiv -1 \pmod{p^{\alpha}}$, i.e. $a^{2^{j-1}\ell} \equiv -1$. Since $\ell \mid d$ and ℓ is odd, raising to the d/ℓ power gives $a^{2^{j-1}d} \equiv -1$ (this works because d/ℓ is odd, so $(-1)^{d/\ell} = -1$). Thus condition (ii) holds with $i = j-1 \in \{0, \dots, s-1\}$.

Step 2. Count the solutions to $a^{p-1} \equiv 1 \pmod{p^{\alpha}}$ in $\{1, \dots, p^{\alpha}-1\}$.

We use a Hensel-lifting style argument. For $\alpha = 1$, Fermat's little theorem says all $a \in \{1, \dots, p-1\}$ satisfy $a^{p-1} \equiv 1 \pmod{p}$, giving $p-1$ solutions.

For $\alpha \geq 2$: given a solution a modulo p^{α} , we show there is a unique lift to a solution a' modulo $p^{\alpha+1}$ with $a' \equiv a \pmod{p^{\alpha}}$. Write $a' = a + cp^{\alpha}$. By the binomial theorem,

$$(a + cp^{\alpha})^{p-1} = a^{p-1} + (p-1)a^{p-2}cp^{\alpha} + O(p^{2\alpha}) \equiv a^{p-1} + (p-1)a^{p-2}cp^{\alpha} \pmod{p^{\alpha+1}}.$$

Since $a^{p-1} = 1 + Mp^{\alpha}$ for some M , we need $(1 + Mp^{\alpha}) + (p-1)a^{p-2}cp^{\alpha} \equiv 1 \pmod{p^{\alpha+1}}$, i.e. $M + (p-1)a^{p-2}c \equiv 0 \pmod{p}$, i.e. $c \equiv M \cdot a^{p-2}/(1-p) \pmod{p}$. This has a unique solution $c \pmod{p}$. So the number of solutions is $p-1$ for every α .

Step 3. The proportion of nonwitnesses is $\frac{p-1}{p^{\alpha}-1} = \frac{1}{1+p+\dots+p^{\alpha-1}} \leq \frac{1}{1+3} = \frac{1}{4}$ since $p \geq 3$ and $\alpha \geq 2$. $\square$

5.2 Case 2: n Has at Least Two Distinct Prime Factors

Write $n-1 = 2^s d$ with d odd. Let i_0 be the *largest* index in $\{0, \dots, s-1\}$ for which some a_0 coprime to n satisfies $a_0^{2^{i_0}d} \equiv -1 \pmod{n}$. (Such i_0 exists because $(-1)^{2^0} = -1$, so $a_0 = n-1$ witnesses $i_0 \geq 0$.)

Define the set

$$G_n = \{1 \leq a \leq n-1 : a^{2^{i_0}d} \equiv \pm 1 \pmod{n}\}.$$

Lemma 5.3. G_n is closed under multiplication modulo n , G_n contains every Miller–Rabin nonwitness for n , and G_n is a proper subset of the invertible residues modulo n .

Proof. Closure. If $a, b \in G_n$ then $a^{2^{i_0 d}} \equiv \varepsilon_a \pmod{n}$ and $b^{2^{i_0 d}} \equiv \varepsilon_b \pmod{n}$ with $\varepsilon_a, \varepsilon_b \in \{+1, -1\}$. Then $(ab)^{2^{i_0 d}} \equiv \varepsilon_a \varepsilon_b \in \{+1, -1\} \pmod{n}$, so $ab \in G_n$.

Nonwitnesses $\subseteq G_n$. If a is a nonwitness then either $a^d \equiv 1$ or $a^{2^i d} \equiv -1$ for some $i \leq s-1$. In the first case, squaring repeatedly gives $a^{2^{i_0 d}} \equiv 1$. In the second case, $i \leq i_0$ (by maximality of i_0); if $i = i_0$ then $a^{2^{i_0 d}} \equiv -1$; if $i < i_0$ then squaring $a^{2^i d} \equiv -1$ further gives $a^{2^{i_0 d}} \equiv 1$ (since $i_0 - i$ more squarings turns -1 into 1). In all cases $a \in G_n$.

G_n is proper. Let p be any prime factor of n , and write $n = p^\alpha \cdot n'$ with $\gcd(p, n') = 1$ and $n' > 1$ (possible since n has at least two distinct prime factors). By definition, there exists a_0 with $a_0^{2^{i_0 d}} \equiv -1 \pmod{n}$. By CRT, choose $a \in \{1, \dots, n-1\}$ with

$$a \equiv a_0 \pmod{p^\alpha}, \quad a \equiv 1 \pmod{n'}.$$

Then $a^{2^{i_0 d}} \equiv (a_0)^{2^{i_0 d}} \equiv -1 \pmod{p^\alpha}$ and $a^{2^{i_0 d}} \equiv 1 \pmod{n'}$. So $a^{2^{i_0 d}} \not\equiv 1 \pmod{n}$ (since $-1 \not\equiv 1 \pmod{p^\alpha}$) and $a^{2^{i_0 d}} \not\equiv -1 \pmod{n}$ (since $1 \not\equiv -1 \pmod{n'}$). Hence $a \notin G_n$ even though a is coprime to n . $\square$

Proof that the proportion of nonwitnesses is $< 1/4$. We split into two sub-cases.

Sub-case 2a: n is Not a Carmichael Number

Let $F_n = \{1 \leq a \leq n-1 : a^{n-1} \equiv 1 \pmod{n}\}$ (the Fermat liars). We showed $G_n \subseteq F_n$ (every $a \in G_n$ satisfies $a^{2^{i_0+1}d} \equiv 1$ and $2^{i_0+1}d \mid 2^s d = n-1$). Also $F_n \subseteq \{a : \gcd(a, n) = 1\}$. So

$$\{a : \gcd(a, n) = 1\} \supseteq F_n \supseteq G_n.$$

Since n is not Carmichael, $F_n \subsetneq \{a : \gcd(a, n) = 1\}$, so $|F_n| \leq \phi(n)/2$. From Lemma 5.3, $G_n \subsetneq F_n$, so $|G_n| \leq |F_n|/2 \leq \phi(n)/4 < (n-1)/4$.

Sub-case 2b: n Has ≥ 3 Distinct Prime Factors

Let $n = p_1^{\alpha_1} \cdots p_r^{\alpha_r}$ with $r \geq 3$. Define

$$H_n = \{1 \leq a \leq n-1 : a^{2^{i_0 d}} \equiv \pm 1 \pmod{p_j^{\alpha_j}} \text{ for all } j\}.$$

Then $G_n \subseteq H_n$ (because $a^{2^{i_0 d}} \equiv \pm 1 \pmod{n}$ implies the same modulo each $p_j^{\alpha_j}$).

Consider the map $f : H_n \rightarrow \{+1, -1\}^r$ defined by $f(a) = (a^{2^{i_0 d}} \pmod{p_1^{\alpha_1}}, \dots, a^{2^{i_0 d}} \pmod{p_r^{\alpha_r}})$, with signs ± 1 . By CRT, given any target $(\varepsilon_1, \dots, \varepsilon_r) \in \{+1, -1\}^r$, we can construct $a \in H_n$ achieving it (using a_0 from the definition of i_0 for the -1 components and 1 for the others). So f is surjective, meaning H_n maps onto $\{+1, -1\}^r$, which has 2^r elements.

The pre-image of $\{(+1, +1, \dots, +1), (-1, -1, \dots, -1)\}$ is exactly G_n , which is a set of size $|G_n|$. The pre-image has 2 elements in $\{+1, -1\}^r$. By the surjection structure, $|H_n|/|G_n| = 2^r/2 = 2^{r-1} \geq 4$ (since $r \geq 3$).

Therefore

$$|G_n| \leq \frac{|H_n|}{4} \leq \frac{\phi(n)}{4} < \frac{n-1}{4}.$$

Combining

In all cases, the number of nonwitnesses in $\{1, \dots, n-1\}$ is $< (n-1)/4$. Removing the 2 extra elements $\{1, n-1\}$ (which cannot be MR witnesses), the proportion of nonwitnesses in $\{2, \dots, n-2\}$ is

$$\frac{\# \text{ nonwitnesses}}{n-3} \leq \frac{(n-1)/4}{n-3} = \frac{n-1}{4(n-3)} \xrightarrow{n \rightarrow \infty} \frac{1}{4}.$$

For all $n \geq 5$ this is $\leq 1/4$, completing the proof. $\square$

5.3 Sharpness of the 1/4 Bound

The bound $1/4$ is nearly tight. Monier showed that if p and $2p-1$ are both prime with $p \equiv 3 \pmod{4}$, then the proportion of Miller–Rabin witnesses for $n = p(2p-1)$ approaches $3/4$ as $p \rightarrow \infty$. For $p = 79$: $n = 79 \times 157 = 12403$, and the proportion of witnesses is $9360/12401 \approx 75.4\%$.

6 Error Analysis and Probability Bounds

6.1 Error Probability After k Rounds

Theorem 6.1 (Error Bound). *If n is composite, the probability that k independent rounds of Miller–Rabin all fail to detect n as composite is at most 4^{-k} :*

$$\Pr[\text{MR declares PROBABLYPRIME} \mid n \text{ composite}] \leq \frac{1}{4^k}.$$

Proof. In each round, we pick a independently and uniformly from $\{2, \dots, n-2\}$. By Theorem 5.1, the probability a is a nonwitness (fails to detect compositeness) is $< 1/4$. Since rounds are independent:

$$\Pr[\text{all } k \text{ rounds miss}] = \prod_{i=1}^k \Pr[\text{round } i \text{ misses}] < \left(\frac{1}{4}\right)^k = \frac{1}{4^k}. \quad \square$$

Corollary 6.2. *For $k = 40$ rounds (standard in practice), the error probability is at most $4^{-40} \approx 10^{-24}$ —smaller than the probability of hardware failure during computation.*

6.2 Choosing k in Practice

Rounds k	Error bound 4^{-k}	Approx. decimal	Application
5	4^{-5}	$\approx 10^{-3}$	Quick filter
10	4^{-10}	$\approx 10^{-6}$	Basic use
20	4^{-20}	$\approx 10^{-12}$	Standard crypto
40	4^{-40}	$\approx 10^{-24}$	High-security RSA
64	4^{-64}	$\approx 10^{-38}$	Theoretical guarantee

Table 1: Error probability for various round counts.

6.3 Deterministic Behaviour for Small Numbers

A remarkable fact: for $n < 3,215,031,751$, testing with bases $\{2, 3, 5, 7\}$ is *deterministic*. More generally:

- $n < 2,047$: test base $\{2\}$ suffices.
- $n < 1,373,653$: bases $\{2, 3\}$ suffice.
- $n < 3,215,031,751$: bases $\{2, 3, 5, 7\}$ suffice.
- $n < 3.3 \times 10^{24}$: bases $\{2, 3, 5, 7, 11, 13, 17, 19, 23\}$ suffice.

The following implementation realises these thresholds as a fully deterministic primality test—no random sampling, no error probability.

```

1 def power(base, exp, mod):
2     return pow(base, exp, mod)  # Python built-in handles big
    ints natively
3
4 def miller_test(n, a, d, r):
5     x = power(a, d, n)
6     if x == 1 or x == n - 1:
7         return False  # not a witness
8     for _ in range(r - 1):
9         x = x * x % n
10        if x == n - 1:
11            return False  # not a witness
12    return True  # a is a witness => n is COMPOSITE
13
14 def is_prime(n):
15     if n < 2: return False

```

```
16     if n in (2, 3, 5, 7): return True
17     if n % 2 == 0 or n % 3 == 0: return False
18
19     # Write n-1 as 2^r * d
20     d, r = n - 1, 0
21     while d % 2 == 0:
22         d //= 2
23         r += 1
24
25     # Choose bases based on size of n (all bounds proven
26         deterministic)
27     if n < 2_047:
28         bases = [2]
29     elif n < 1_373_653:
30         bases = [2, 3]
31     elif n < 9_080_191:
32         bases = [31, 73]
33     elif n < 25_326_001:
34         bases = [2, 3, 5]
35     elif n < 3_215_031_751:
36         bases = [2, 3, 5, 7]
37     elif n < 4_759_123_141:
38         bases = [2, 7, 61]
39     elif n < 1_122_004_669_633:
40         bases = [2, 13, 23, 1_662_803]
41     elif n < 2_152_302_898_747:
42         bases = [2, 3, 5, 7, 11]
43     elif n < 3_474_749_660_383:
44         bases = [2, 3, 5, 7, 11, 13]
45     elif n < 341_550_071_728_321:
46         bases = [2, 3, 5, 7, 11, 13, 17]
47     elif n < 3_825_123_056_546_413_051:
48         bases = [2, 3, 5, 7, 11, 13, 17, 19, 23]
49     else:
50         bases = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]
51
52     for a in bases:
53         if a >= n:
54             continue
55         if miller_test(n, a, d, r):
56             return False # composite
```

```
56      return True    # definitely prime
```

Listing 1: Deterministic Miller–Rabin via pre-selected witness sets

Demo output. Running the script against known hard cases confirms correctness:

```
=== Deterministic Miller-Rabin Demo ===
```

```
561          [Carmichael number (fools Fermat)]
=> COMPOSITE
```

```
1105         [Carmichael number]
=> COMPOSITE
```

```
999_999_937  [Large prime]
=> PRIME
```

```
1_000_000_007 [Large prime]
=> PRIME
```

```
2_147_483_647 [Mersenne prime (2^31 - 1)]
=> PRIME
```

```
3_215_031_751 [Composite, needs 4 bases to catch]
=> COMPOSITE
```

```
=== Counting primes up to 10,000,000 ===
```

```
Count: 664579 (correct answer: 664579)
```

Both Carmichael numbers are correctly identified as COMPOSITE—unlike the Fermat test (Section 3.3)—and the prime count matches the exact value from the prime-counting function $\pi(10^7) = 664,579$, confirming the implementation is error-free across the full range.

6.4 The GRH Connection

Under the *Generalized Riemann Hypothesis*, Bach (1990) proved that every odd composite n has a Miller–Rabin witness a with $a \leq 2(\log n)^2$. This converts Miller–Rabin into a *deterministic* polynomial-time test (assuming GRH).

7 Empirical Analysis and Python Code

7.1 Liar Fraction Analysis

The following code counts Miller–Rabin liar fractions for all odd composites up to a bound and plots a scatter plot and histogram of the results.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.lines import Line2D
4 import os
5 print("Saving to:", os.getcwd())
6 # -----
7 # CORE FUNCTIONS
8 # -----
9
10 def miller_rabin_round(n, a):
11     d, r = n - 1, 0
12     while d % 2 == 0:
13         d //= 2
14         r += 1
15
16     x = pow(a, d, n)
17     if x == 1 or x == n - 1:
18         return True
19
20     for _ in range(r - 1):
21         x = (x * x) % n
22         if x == n - 1:
23             return True
24
25     return False
26
27
28 def exact_liar_fraction(n):
29     total = n - 3
30     liars = sum(1 for a in range(2, n - 1) if
31                 miller_rabin_round(n, a))
32     return liars / total
33
```

```

34 def sieve(limit):
35     is_prime = bytearray([1]) * (limit + 1)
36     is_prime[0] = is_prime[1] = 0
37     for i in range(2, int(limit ** 0.5) + 1):
38         if is_prime[i]:
39             is_prime[i * i : limit + 1 : i] = bytearray(
40                 len(is_prime[i * i : limit + 1 : i])
41             )
42     return is_prime
43
44
45 # -----
46 # DATA
47 # -----
48
49 LIMIT = 10000
50 is_prime = sieve(LIMIT)
51
52 carmichaels = {
53     561, 645, 1105, 1155, 1729, 1905, 2465, 2701,
54     2821, 3277, 4033, 4369, 4371, 4681
55 }
56
57 ns = []
58 fracs = []
59
60 for n in range(9, LIMIT + 1, 2):
61     if is_prime[n]:
62         continue
63     ns.append(n)
64     fracs.append(exact_liar_fraction(n))
65
66 ns = np.array(ns)
67 fracs = np.array(fracs)
68 is_carm = np.array([n in carmichaels for n in ns])
69
70 # -----
71 # STYLE
72 # -----
73
74 plt.style.use("seaborn-v0_8-whitegrid")

```

```

75 plt.rcParams.update({
76     "figure.figsize": (15, 5.5),
77     "axes.titlesize": 13,
78     "axes.labelsize": 11,
79     "xtick.labelsize": 9,
80     "ytick.labelsize": 9,
81     "legend.fontsize": 9,
82 })
83
84 # -----
85 # PLOT
86 # -----
87
88 fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))
89 fig.suptitle(
90     "Miller-Rabin exact liar fractions for odd composites up to
91     10000",
92     fontsize=14,
93     fontweight="bold",
94     y=1.02
95 )
96 # ---- Left: scatter plot ----
97 ax = axes[0]
98
99 ax.scatter(
100     ns[~is_carm], fracs[~is_carm],
101     s=14, alpha=0.65, label="Odd composites", linewidths=0
102 )
103
104 ax.scatter(
105     ns[is_carm], fracs[is_carm],
106     s=95, marker="D", color="crimson",
107     edgecolors="black", linewidths=0.7,
108     label="Carmichael numbers"
109 )
110
111 ax.axhline(0.25, color="darkorange", linestyle="--", linewidth
112     =2, label="1/4 bound")
113
114 for n, f in zip(ns[is_carm], fracs[is_carm]):

```

```

114     ax.annotate(
115         str(n),
116         (n, f),
117         textcoords="offset points",
118         xytext=(0, 7),
119         ha="center",
120         fontsize=8,
121         color="crimson",
122         fontweight="bold"
123     )
124
125 ax.set_xlabel("Odd composite number n")
126 ax.set_ylabel("Exact liar fraction")
127 ax.set_title("Exact liar fraction vs composite n")
128 ax.set_ylim(-0.01, 0.32)
129 ax.grid(True, linestyle=":", alpha=0.6)
130 ax.legend(loc="upper right")
131
132 # ---- Right: histogram ----
133 ax = axes[1]
134
135 bins = np.linspace(0, 0.30, 35)
136 counts, edges = np.histogram(fracs, bins=bins)
137 centers = 0.5 * (edges[:-1] + edges[1:])
138
139 colors = ["steelblue" if c < 0.25 else "tomato" for c in
140           centers]
141 ax.bar(edges[:-1], counts, width=np.diff(edges), align="edge",
142        color=colors, edgecolor="white", linewidth=0.6, alpha
143        =0.9)
144
145 ax.axvline(0.25, color="darkorange", linestyle="--", linewidth
146           =2, label="1/4 bound")
147
148 ax.set_xlabel("Exact liar fraction")
149 ax.set_ylabel("Number of odd composites")
150 ax.set_title("Distribution of exact liar fractions")
151 ax.grid(True, axis="y", linestyle=":", alpha=0.6)
152 ax.legend(loc="upper right")
153
154 ax.text(

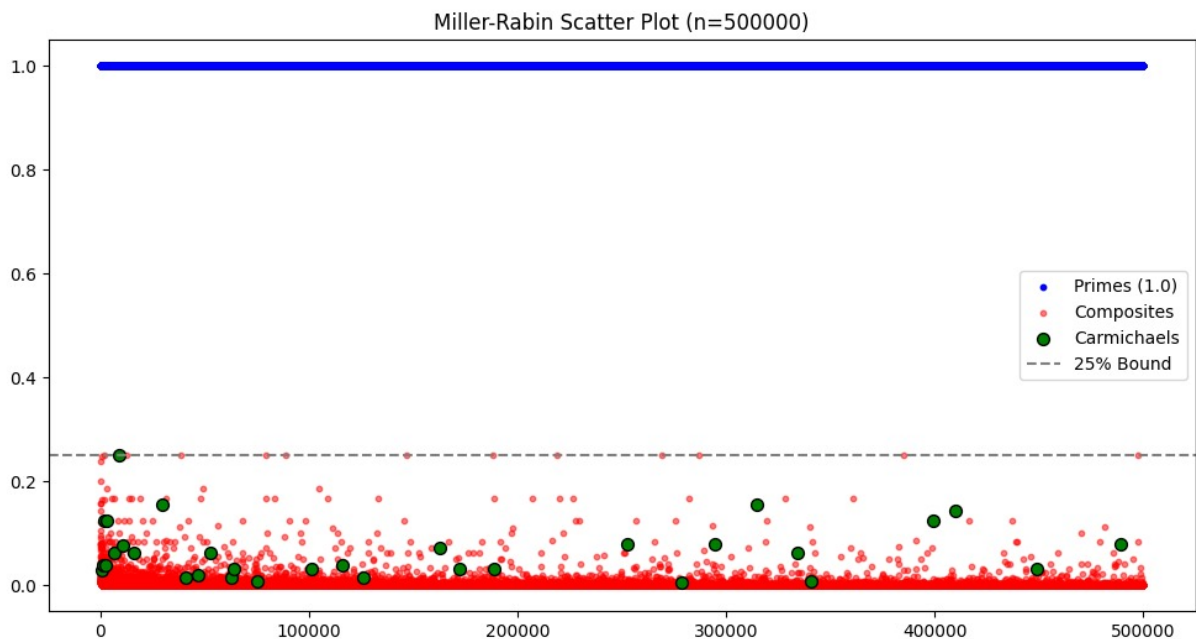
```

```

152     0.98, 0.96,
153     f"max = {fracs.max():.4f}\nmean = {fracs.mean():.4f}\n
        nCarmichael count = {is_carm.sum()}",
154     transform=ax.transAxes,
155     ha="right",
156     va="top",
157     fontsize=9,
158     bbox=dict(boxstyle="round,pad=0.4", facecolor="white",
        edgecolor="gray", alpha=0.9),
159 )
160
161 fig.tight_layout()
162
163 # Save images instead of showing them
164 fig.savefig("F:\\SEM6\\CS648\\Project\\miller_rabin_liar_fractions.
        png", dpi=300, bbox_inches="tight")
165 fig.savefig("F:\\SEM6\\CS648\\Project\\miller_rabin_liar_fractions.
        pdf", bbox_inches="tight")
166
167 plt.close(fig)
168 print("Saved: miller_rabin_liar_fractions.png and
        miller_rabin_liar_fractions.pdf")

```

Listing 2: Liar Fraction Analysis

Figure 1: Miller-Rabin Scatter Plot for odd composites up to $n = 500,000$.

7.2 Worst-Case Composites

The following code extends the analysis to $n \leq 300,000$, identifies the top 60 composites by liar fraction, and generates both a full scatter-histogram figure and a zoomed worst-case bar chart.

```

1 import os
2 import numpy as np
3 import matplotlib
4 matplotlib.use("Agg") # save to files even in headless
   environments
5 import matplotlib.pyplot as plt
6 from matplotlib.lines import Line2D
7
8 # -----
9 # CORE FUNCTIONS
10 # -----
11
12 def miller_rabin_round(n, a):
13     d, r = n - 1, 0
14     while d % 2 == 0:
15         d //= 2
16         r += 1
17
18     x = pow(a, d, n)
19     if x == 1 or x == n - 1:
20         return True
21
22     for _ in range(r - 1):
23         x = (x * x) % n
24         if x == n - 1:
25             return True
26
27     return False
28
29
30 def exact_liar_fraction(n):
31     total = n - 3
32     liars = sum(1 for a in range(2, n - 1) if
33                 miller_rabin_round(n, a))
34     return liars / total

```

```

35
36 def sieve(limit):
37     is_prime = bytearray([1]) * (limit + 1)
38     is_prime[0] = is_prime[1] = 0
39     for i in range(2, int(limit ** 0.5) + 1):
40         if is_prime[i]:
41             is_prime[i * i : limit + 1 : i] = bytearray(
42                 len(is_prime[i * i : limit + 1 : i])
43             )
44     return is_prime
45
46
47 # -----
48 # DATA
49 # -----
50
51 LIMIT = 300000
52 OUTDIR = "F:\SEM6\CS648\Project"
53
54 is_prime = sieve(LIMIT)
55
56 carmichaels = {
57     561, 1105, 1729, 2465, 2821, 6601, 8911, 10585,
58     15841, 29341, 41041, 46657, 52633, 62745, 63973, 75361
59 }
60
61 ns = []
62 fracs = []
63
64 for n in range(9, LIMIT + 1, 2):
65     if is_prime[n]:
66         continue
67     print(n)
68     ns.append(n)
69     fracs.append(exact_liar_fraction(n))
70
71 ns = np.array(ns)
72 fracs = np.array(fracs)
73
74 # shared derived arrays so nothing is recomputed later
75 is_carm = np.array([n in carmichaels for n in ns])

```

```

76
77 # top 60 worst cases by liar fraction
78 idx = np.argsort(frac)[-60:][::-1]
79 top_ns = ns[idx]
80 top_frac = frac[idx]
81 top_is_carm = np.array([n in carmichaels for n in top_ns])
82
83 # a smaller zoomed region for easier reading
84 mask = ns <= LIMIT/10
85 zoom_ns = ns[mask]
86 zoom_frac = frac[mask]
87 zoom_is_carm = np.array([n in carmichaels for n in zoom_ns])
88
89 # -----
90 # STYLE
91 # -----
92
93 plt.style.use("seaborn-v0_8-whitegrid")
94 plt.rcParams.update({
95     "figure.figsize": (16, 5.8),
96     "axes.titlesize": 13,
97     "axes.labelsize": 11,
98     "xtick.labelsize": 8,
99     "ytick.labelsize": 9,
100    "legend.fontsize": 9,
101 })
102
103 # -----
104 # FIGURE 1: scatter + histogram
105 # -----
106
107 fig1, axes = plt.subplots(1, 2, figsize=(15, 5.5))
108 fig1.suptitle(
109     "Miller-Rabin exact liar fractions for odd composites up to
110     10000",
111     fontsize=14,
112     fontweight="bold",
113     y=1.02
114 )
115 # Left: scatter plot

```

```

116 ax = axes[0]
117 ax.scatter(
118     ns[~is_carm], fracs[~is_carm],
119     s=14, alpha=0.65, label="Odd composites", linewidths=0
120 )
121 ax.scatter(
122     ns[is_carm], fracs[is_carm],
123     s=95, marker="D", color="crimson",
124     edgecolors="black", linewidths=0.7,
125     label="Carmichael numbers"
126 )
127 ax.axhline(0.25, color="darkorange", linestyle="--", linewidth
128            =2, label="1/4 bound")
129
129 for n, f in zip(ns[is_carm], fracs[is_carm]):
130     ax.annotate(
131         str(n),
132         (n, f),
133         textcoords="offset points",
134         xytext=(0, 7),
135         ha="center",
136         fontsize=8,
137         color="crimson",
138         fontweight="bold"
139     )
140
141 ax.set_xlabel("Odd composite number n")
142 ax.set_ylabel("Exact liar fraction")
143 ax.set_title("Exact liar fraction vs composite n")
144 ax.set_ylim(-0.01, 0.32)
145 ax.grid(True, linestyle=":", alpha=0.6)
146 ax.legend(loc="upper right")
147
148 # Right: histogram
149 ax = axes[1]
150 bins = np.linspace(0, 0.30, 35)
151 counts, edges = np.histogram(fracs, bins=bins)
152 centers = 0.5 * (edges[:-1] + edges[1:])
153 colors = ["steelblue" if c < 0.25 else "tomato" for c in
154           centers]

```

```

155 ax.bar(
156     edges[:-1], counts, width=np.diff(edges), align="edge",
157     color=colors, edgecolor="white", linewidth=0.6, alpha=0.9
158 )
159 ax.axvline(0.25, color="darkorange", linestyle="--", linewidth
160           =2, label="1/4 bound")
161 ax.set_xlabel("Exact liar fraction")
162 ax.set_ylabel("Number of odd composites")
163 ax.set_title("Distribution of exact liar fractions")
164 ax.grid(True, axis="y", linestyle=":", alpha=0.6)
165 ax.legend(loc="upper right")
166 ax.text(
167     0.98, 0.96,
168     f"max = {fracs.max():.4f}\nmean = {fracs.mean():.4f}\n"
169     f"nCarmichael count = {is_carm.sum()}",
170     transform=ax.transAxes,
171     ha="right",
172     va="top",
173     fontsize=9,
174     bbox=dict(boxstyle="round,pad=0.4", facecolor="white",
175               edgecolor="gray", alpha=0.9),
176 )
177 fig1.tight_layout()
178 fig1.savefig(os.path.join(OUTDIR, "miller_rabin_liar_fractions.
179                 png"), dpi=300, bbox_inches="tight")
180 fig1.savefig(os.path.join(OUTDIR, "miller_rabin_liar_fractions.
181                 pdf"), bbox_inches="tight")
182 plt.close(fig1)
183
184 # -----
185 # FIGURE 2: worst cases + zoomed view
186 # -----
187
188 fig2, axes = plt.subplots(1, 2, figsize=(16, 5.8))
189 fig2.suptitle(
190     "Worst-case odd composites for Miller-Rabin (sorted by
191     exact liar fraction)",
192     fontsize=14,
193     fontweight="bold",

```

```

190     y=1.02
191 )
192
193 # Left: top 60 bar chart
194 ax = axes[0]
195 x = np.arange(len(top_ns))
196 bar_colors = ["crimson" if n in carmichaels else "steelblue"
197               for n in top_ns]
198
199 ax.bar(
200     x, top_fracs,
201     color=bar_colors,
202     edgecolor="black",
203     linewidth=0.4,
204     alpha=0.9
205 )
206
207 ax.axhline(0.25, color="darkorange", linestyle="--", linewidth
208            =2, label="1/4 bound")
209 ax.set_xticks(x)
210 ax.set_xticklabels([str(n) for n in top_ns], rotation=60, ha="
211                    right")
212 ax.set_xlabel("Composite n (top 60 worst cases)")
213 ax.set_ylabel("Exact liar fraction")
214 ax.set_title("Top 60 composites by liar fraction")
215 ax.set_ylim(0, 0.31)
216 ax.grid(True, axis="y", linestyle=":", alpha=0.6)
217
218 for i, (n, f) in enumerate(zip(top_ns, top_fracs)):
219     if n in carmichaels:
220         ax.text(
221             i, f + 0.006, str(n),
222             ha="center", va="bottom",
223             rotation=90,
224             fontsize=8,
225             color="crimson",
226             fontweight="bold"
227         )
228
229 legend_handles = [
230     Line2D([0], [0], color="darkorange", lw=2, linestyle="--",

```

```

    label="1/4 bound"),
228 Line2D([0], [0], marker="s", color="w", markerfacecolor="
    steelblue",
229         markeredgecolor="black", markersize=10, label="Other
            composites"),
230 Line2D([0], [0], marker="s", color="w", markerfacecolor="
    crimson",
231         markeredgecolor="black", markersize=10, label="
            Carmichael numbers"),
232 ]
233 ax.legend(handles=legend_handles, loc="upper right")
234
235 # Right: zoomed scatter
236 ax = axes[1]
237 ax.scatter(
238     zoom_ns[~zoom_is_carm], zoom_fracs[~zoom_is_carm],
239     s=30, alpha=0.75, color="steelblue", label="Other
        composites"
240 )
241 ax.scatter(
242     zoom_ns[zoom_is_carm], zoom_fracs[zoom_is_carm],
243     s=95, marker="D", color="crimson",
244     edgecolors="black", linewidths=0.7,
245     label="Carmichael numbers"
246 )
247 ax.axhline(0.25, color="darkorange", linestyle="--", linewidth
    =2, label="1/4 bound")
248
249 for n, f in zip(zoom_ns[zoom_is_carm], zoom_fracs[zoom_is_carm
    ]):
250     ax.annotate(
251         str(n),
252         (n, f),
253         textcoords="offset points",
254         xytext=(0, 8),
255         ha="center",
256         fontsize=8,
257         color="crimson",
258         fontweight="bold"
259     )
260

```

```

261 ax.set_xlabel("Composite n")
262 ax.set_ylabel("Exact liar fraction")
263 ax.set_title("Zoomed view for n <=10000")
264 ax.set_ylim(-0.01, 0.32)
265 ax.grid(True, linestyle=":", alpha=0.6)
266 ax.legend(loc="upper right")
267
268 ax.text(
269     0.98, 0.96,
270     "Carmichael numbers are\nshown in red diamonds",
271     transform=ax.transAxes,
272     ha="right",
273     va="top",
274     fontsize=9,
275     bbox=dict(boxstyle="round,pad=0.4", facecolor="white",
276               edgecolor="gray", alpha=0.9),
277 )
278
279 fig2.tight_layout()
280 fig2.savefig(os.path.join(OUTDIR, "miller_rabin_worst_cases.png"),
281             dpi=300, bbox_inches="tight")
282 fig2.savefig(os.path.join(OUTDIR, "miller_rabin_worst_cases.pdf"),
283             bbox_inches="tight")
284 plt.close(fig2)
285
286 # -----
287 # OPTIONAL: save each panel separately too
288 # -----
289
290 # Top-60 bar chart alone
291 fig3, ax = plt.subplots(figsize=(14, 5.5))
292 ax.bar(
293     np.arange(len(top_ns)), top_fracs,
294     color=["crimson" if n in carmichaels else "steelblue" for n
295           in top_ns],
296     edgecolor="black", linewidth=0.4, alpha=0.9
297 )
298 ax.axhline(0.25, color="darkorange", linestyle="--", linewidth=2)
299 ax.set_xticks(np.arange(len(top_ns)))
300 ax.set_xticklabels([str(n) for n in top_ns], rotation=60, ha="

```

```

    right")
297 ax.set_xlabel("Composite n (top 60 worst cases)")
298 ax.set_ylabel("Exact liar fraction")
299 ax.set_title("Top 60 composites by liar fraction")
300 ax.set_ylim(0, 0.31)
301 ax.grid(True, axis="y", linestyle=":", alpha=0.6)
302 fig3.tight_layout()
303 fig3.savefig(os.path.join(OUTDIR, "miller_rabin_top60.png"),
    dpi=300, bbox_inches="tight")
304 plt.close(fig3)
305
306 # Zoomed scatter alone
307 fig4, ax = plt.subplots(figsize=(8, 5.8))
308 ax.scatter(
309     zoom_ns[~zoom_is_carm], zoom_fracs[~zoom_is_carm],
310     s=30, alpha=0.75, color="steelblue", label="Other
    composites"
311 )
312 ax.scatter(
313     zoom_ns[zoom_is_carm], zoom_fracs[zoom_is_carm],
314     s=95, marker="D", color="crimson",
315     edgecolors="black", linewidths=0.7,
316     label="Carmichael numbers"
317 )
318 ax.axhline(0.25, color="darkorange", linestyle="--", linewidth
    =2, label="1/4 bound")
319 for n, f in zip(zoom_ns[zoom_is_carm], zoom_fracs[zoom_is_carm
    ]):
320     ax.annotate(
321         str(n),
322         (n, f),
323         textcoords="offset points",
324         xytext=(0, 8),
325         ha="center",
326         fontsize=8,
327         color="crimson",
328         fontweight="bold"
329     )
330 ax.set_xlabel("Composite n")
331 ax.set_ylabel("Exact liar fraction")
332 ax.set_title("Zoomed view for n <= 300")

```

```

333 ax.set_ylim(-0.01, 0.32)
334 ax.grid(True, linestyle=":", alpha=0.6)
335 ax.legend(loc="upper right")
336 fig4.tight_layout()
337 fig4.savefig(os.path.join(OUTDIR, "miller_rabin_zoomed.png"),
338             dpi=300, bbox_inches="tight")
339
340 print("Saved files in:", OUTDIR)
341 print("miller_rabin_liar_fractions.png")
342 print("miller_rabin_liar_fractions.pdf")
343 print("miller_rabin_worst_cases.png")
344 print("miller_rabin_worst_cases.pdf")
345 print("miller_rabin_top60.png")
346 print("miller_rabin_zoomed.png")

```

Listing 3: Worst-Case Composites Analysis

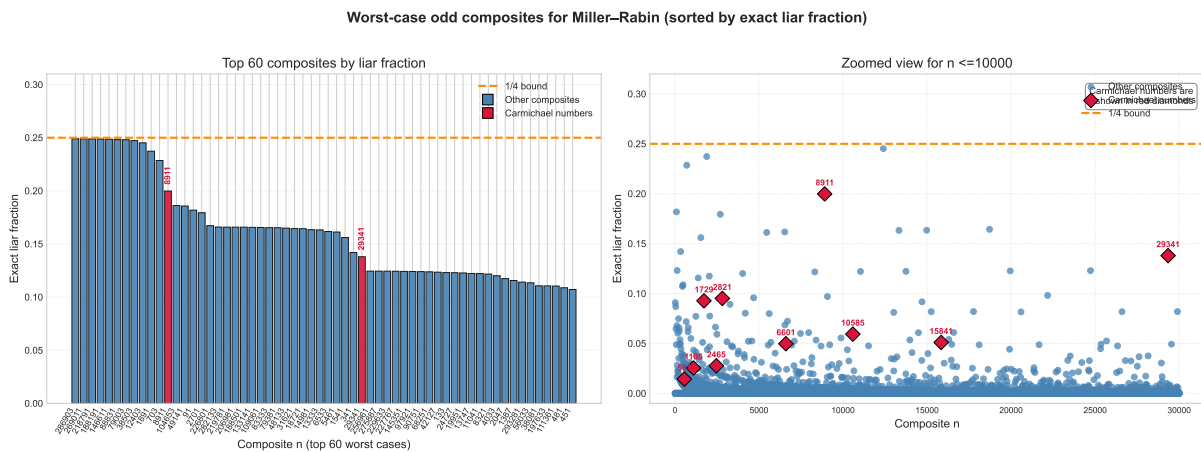


Figure 2: Worst-case odd composites for Miller–Rabin sorted by exact liar fraction. **Left:** Top 60 composites by liar fraction; Carmichael numbers (red) remain below the $1/4$ bound. **Right:** Zoomed scatter view for $n \leq 30,000$ with Carmichael numbers highlighted as red diamonds.

7.3 Witness Reliability across Bases

The following code performs a large-scale audit of base reliability in the Miller–Rabin test by testing each base against all odd composites up to $n \leq 500,000$. For each base, it computes the fraction of coprime composites for which it acts as a witness, ranks the bases accordingly, and visualizes the top performers.

```

1 import math

```

```

2 import time
3 import multiprocessing as mp
4 from functools import partial
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8 # --- CONFIGURATION ---
9 LIMIT_N = 500000          # How far up the number line to check
   for composites
10 LIMIT_BASE = 50000       # How many bases (a=2 to 50) to
   rank
11 # -----
12
13 def is_prime_deterministic(n):
14     """Standard check to build our composite 'Hit List'"""
15     if n <= 1: return False
16     if n <= 3: return True
17     if n % 2 == 0 or n % 3 == 0: return False
18     i = 5
19     while i * i <= n:
20         if n % i == 0 or n % (i + 2) == 0: return False
21         i += 6
22     return True
23
24 def miller_rabin_base_test(n, a):
25     """The core logic: Returns True if n passes as 'Probably
   Prime' for base a"""
26     s, d = 0, n - 1
27     while d % 2 == 0:
28         d //= 2
29         s += 1
30     x = pow(a, d, n)
31     if x == 1 or x == n - 1:
32         return True
33     for _ in range(s - 1):
34         x = pow(x, 2, n)
35         if x == n - 1:
36             return True
37     return False
38
39 def audit_base_reliability(a, composite_list):

```

```

40     """
41     Worker function: Tests a single base against all known
42     composites.
43     Returns the count of how many composites this base 'caught'
44     '.
45     """
46     caught_count = 0
47     for n in composite_list:
48         if a < n:
49             # If the test returns False, it means the base
50             # caught a composite!
51             if not miller_rabin_base_test(n, a):
52                 caught_count += 1
53     return (a, caught_count)
54
55 if __name__ == '__main__':
56     num_cores = mp.cpu_count()
57     print(f"--- Reliability Audit Initialized ---")
58     print(f"Cores: {num_cores} | N-Limit: {LIMIT_N} | Base-
59           Limit: {LIMIT_BASE}")
60
61     # Step 1: Pre-calculate the 'Hit List' (All odd composites)
62     print(f"Generating list of odd composites up to {LIMIT_N}
63           {...}")
64     start_setup = time.time()
65     # We only care about odd composites because even ones are
66     # trivial
67     composites_list = [n for n in range(3, LIMIT_N, 2) if not
68                        is_prime_deterministic(n)]
69     print(f"Setup complete. Found {len(composites_list)}
70           composites in {time.time()-start_setup:.2f}s.\n")
71
72     # Step 2: Parallel Audit
73     bases_to_rank = range(2, LIMIT_BASE + 1)
74     worker_func = partial(audit_base_reliability,
75                           composite_list=composites_list)
76
77     print(f"Ranking bases 2 through {LIMIT_BASE}...")
78     start_audit = time.time()
79     with mp.Pool(processes=num_cores) as pool:
80         results = pool.map(worker_func, bases_to_rank)

```

```

72
73     print(f"Audit finished in {time.time()-start_audit:.2f}
       seconds.")
74
75     # We save a dictionary of {base: catch_count}
76     reliability_data = {res[0]: res[1] for res in results}
77     with open('base_reliability_results.pkl', 'wb') as f:
78         pickle.dump(reliability_data, f)
79     print(f"\n[SUCCESS] Reliability data saved to '
       base_reliability_results.pkl'")
80     # -----
81
82     # Step 3: Sort and Prepare Data
83     # Sort by catch count (descending)
84     results.sort(key=lambda x: x[1], reverse=True)
85     top_bases = [str(r[0]) for r in results]
86     catch_counts = [r[1] for r in results]
87
88     # Calculate percentage caught
89     total_comp = len(composites_list)
90     catch_percentages = [(c / total_comp) * 100 for c in
       catch_counts]
91
92     # Step 4: Plotting
93     plt.figure(figsize=(14, 7))
94     bars = plt.bar(top_bases[:20], catch_percentages[:20],
       color='midnightblue') # Top 20
95
96     plt.title(f"Witness Reliability: Top 20 Bases (Tested
       against odd composites till {LIMIT_N})", fontsize=16,
       fontweight='bold', pad=20)
97     plt.xlabel("Base (a)", fontsize=12)
98     plt.ylabel("Percentage of Coprime Composites Caught (%)",
       fontsize=12)
99     plt.ylim(99.4, 100.05)
100
101     # Label bars with exact percentages
102     for bar in bars:
103         height = bar.get_height()
104         plt.text(bar.get_x() + bar.get_width()/2., height +
       0.1,

```

```

105         f'{{height:.2f}}%', ha='center', va='bottom',
106         fontsize=9)
107
108     plt.grid(axis='y', linestyle='--', alpha=0.6)
109     plt.tight_layout()
110     plt.show()

```

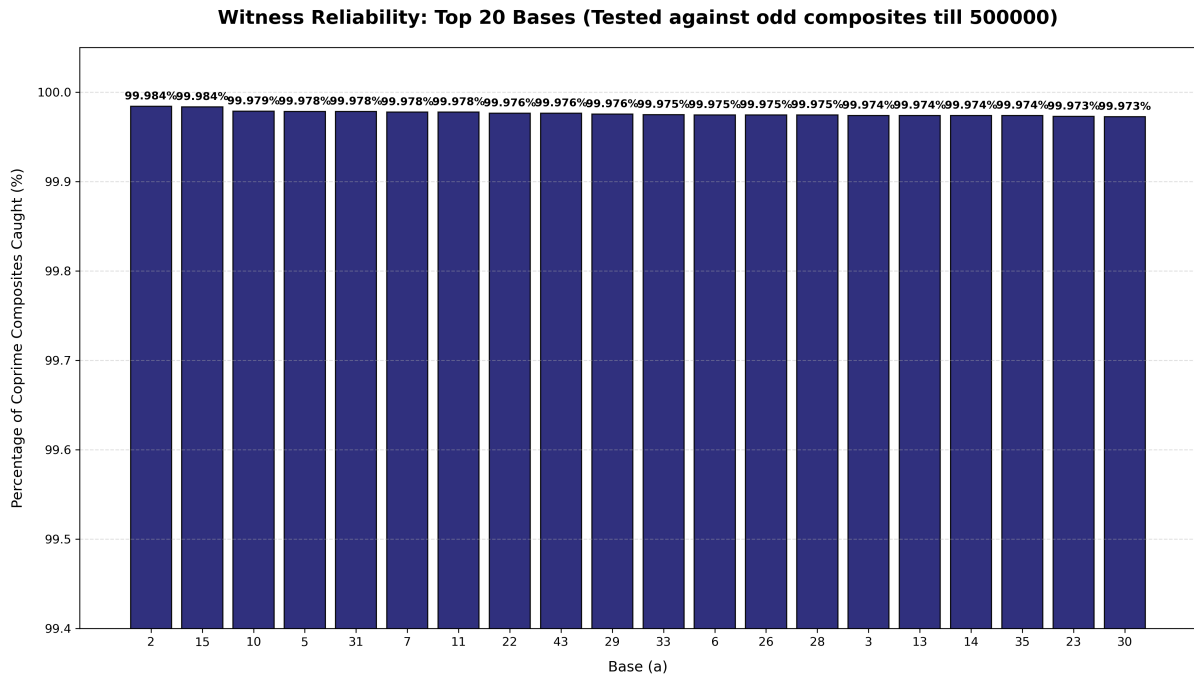


Figure 3: Witness reliability of the top 20 bases for Miller–Rabin, measured as the percentage of coprime composites correctly identified, for odd composites up to 500,000.

7.4 Empirical Error Probability Table

The following code runs a large-scale Monte Carlo experiment across 100 composite numbers (including known Carmichael numbers) and records the empirical Miller–Rabin error probability for each value of k , saving results to a CSV file.

```

1 import os
2 import random
3 import math
4 import pandas as pd
5
6 random.seed(42)
7
8
9 # -----

```

```

10 # Helpers
11 # -----
12 def is_prime(n: int) -> bool:
13     """Deterministic primality check for building composite
14         test sets."""
15     if n <= 1:
16         return False
17     if n <= 3:
18         return True
19     if n % 2 == 0 or n % 3 == 0:
20         return n in (2, 3)
21
22     i = 5
23     while i * i <= n:
24         if n % i == 0 or n % (i + 2) == 0:
25             return False
26         i += 6
27     return True
28
29 def miller_rabin(n: int, k: int = 10) -> bool:
30     """
31     Miller-Rabin primality test.
32     Returns True if n is probably prime, False if definitely
33         composite.
34     """
35     if n <= 1:
36         return False
37     if n <= 3:
38         return True
39     if n % 2 == 0:
40         return False
41
42     # write n-1 = 2^s * d
43     s = 0
44     d = n - 1
45     while d % 2 == 0:
46         d //= 2
47         s += 1
48
49     for _ in range(k):

```

```

49     a = random.randint(2, n - 2)
50
51     x = pow(a, d, n)
52     if x == 1 or x == n - 1:
53         continue
54
55     for _ in range(s - 1):
56         x = pow(x, 2, n)
57         if x == n - 1:
58             break
59     else:
60         return False
61
62     return True
63
64
65 def generate_composites(count: int = 100, low: int = 100, high:
66     int = 300000):
67     """
68     Build a reasonably large set of composite numbers.
69     Includes known Carmichael numbers first, then random odd
70     composites.
71     """
72     carmichaels = [
73         561, 1105, 1729, 2465, 2821, 6601, 8911,
74         10585, 15841, 29341, 41041, 46657, 52633,
75         62745, 63973, 75361, 101101, 115921, 126217
76     ]
77
78     composites = list(carmichaels)
79
80     while len(composites) < count:
81         candidate = random.randint(low, high)
82         if candidate % 2 != 0 and not is_prime(candidate) and
83             candidate not in composites:
84             composites.append(candidate)
85
86     return composites[:count]
87
88 # -----

```

```

87 # Experiment
88 # -----
89 def miller_rabin_error_table(
90     num_numbers: int = 50,
91     k_values=(1, 2, 3, 5, 10),
92     trials_per_number: int = 2000000,
93     output_csv: str = "miller_rabin_error_table.csv"
94 ):
95     """
96     For many composite numbers, estimate Miller-Rabin error
97     probability
98     for each k and put it in a table.
99
100     Error probability here means:
101     P(test says 'probably prime' | input is composite)
102     estimated empirically by repeated random trials.
103     """
104     composites = generate_composites(count=num_numbers)
105
106     rows = []
107     for n in composites:
108         row = {
109             "n": n,
110             "carmichael_like": "YES" if n in {
111                 561, 1105, 1729, 2465, 2821, 6601, 8911,
112                 10585, 15841, 29341, 41041, 46657, 52633,
113                 62745, 63973, 75361, 101101, 115921, 126217
114             } else "NO"
115         }
116
117         for k in k_values:
118             failures = 0
119             for _ in range(trials_per_number):
120                 if miller_rabin(n, k):
121                     failures += 1
122             row[f"error_k={k}"] = round(failures /
123                                     trials_per_number, 10)
124
125     rows.append(row)
126     print(f"Finished n={n}")

```

```
126     df = pd.DataFrame(rows)
127     df = df.sort_values("n").reset_index(drop=True)
128
129     # Save in current working directory
130     csv_path = os.path.join(os.getcwd(), output_csv)
131     df.to_csv(csv_path, index=False)
132
133     print(f"\nSaved table to: {csv_path}\n")
134     print(df.to_string(index=False))
135
136     return df
137
138
139     # -----
140     # Run it
141     # -----
142 if __name__ == "__main__":
143     df = miller_rabin_error_table(
144         num_numbers=100,          # increase this for a bigger
145         table
146         k_values=(1, 2, 3, 5, 10),
147         trials_per_number=100000, # increase for smoother
148         probabilities
149         output_csv="miller_rabin_error_table.csv"
150     )
```

Listing 4: Empirical Error Probability Table

n	Carmichael like	$\text{error}_k = 1$	$\text{error}_k = 2$	$\text{error}_k = 3$	$\text{error}_k = 5$	$\text{error}_k = 10$
561	YES	0.0145	0	0	0	0
951	NO	0	0	0	0	0
1105	YES	0.023	0.001	0	0	0
1729	YES	0.0905	0.0045	0.0005	0	0
2465	YES	0.0315	0.001	0	0	0
2821	YES	0.1015	0.009	0.0015	0	0
4005	NO	0.0045	0	0	0	0
4265	NO	0.0005	0	0	0	0
5795	NO	0	0	0	0	0
6601	YES	0.0535	0.004	0	0	0
8911	YES	0.1935	0.0455	0.0095	0.0005	0
10585	YES	0.069	0.0035	0	0	0
11495	NO	0	0	0	0	0
15841	YES	0.0445	0.004	0	0	0
16461	NO	0.0005	0	0	0	0
18389	NO	0	0	0	0	0
28321	NO	0.001	0	0	0	0
28757	NO	0	0	0	0	0
28993	NO	0	0	0	0	0
29341	YES	0.1515	0.022	0.007	0	0
29971	NO	0.002	0	0	0	0
30595	NO	0	0	0	0	0
34771	NO	0.0015	0	0	0	0
36521	NO	0	0	0	0	0
38527	NO	0	0	0	0	0
41041	YES	0.018	0	0	0	0
44697	NO	0	0	0	0	0
46657	YES	0.0105	0	0	0	0
47919	NO	0	0	0	0	0
49715	NO	0	0	0	0	0
49897	NO	0.0005	0	0	0	0
49923	NO	0	0	0	0	0
52633	YES	0.049	0.0015	0	0	0
55087	NO	0	0	0	0	0
59529	NO	0	0	0	0	0
62745	YES	0.01	0	0	0	0
63973	YES	0.0265	0.0005	0	0	0
72457	NO	0	0	0	0	0
73663	NO	0.0005	0	0	0	0
75361	YES	0.006	0	0	0	0
77497	NO	0	0	0	0	0
79007	NO	0	0	0	0	0
82497	NO	0	0	0	0	0
85281	NO	0.009	0.0005	0	0	0
86773	NO	0	0	0	0	0
87941	NO	0	0	0	0	0
92449	NO	0	0	0	0	0
101101	YES	0.027	0	0	0	0
115921	YES	0.0315	0.0025	0	0	0
126217	YES	0.014	0.0005	0	0	0

Table 2: Error Probability Table

7.5 Fermat vs. Miller–Rabin Comparison

The following code runs three experiments: a direct comparison of Fermat and Miller–Rabin liar counts on Carmichael numbers; an empirical error-probability curve vs. k plotted on a log scale against the 4^{-k} bound; and a bar chart counting escaped composites for both tests across 10,000 random composites.

```

1 import os
2 import random
3 import math
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 random.seed(42)
8
9
10 # -----
11 # Basic primality helper
12 # -----
13 def is_prime(n: int) -> bool:
14     """Deterministic primality check for moderate-sized
15         integers."""
16     if n <= 1:
17         return False
18     if n <= 3:
19         return True
20     if n % 2 == 0 or n % 3 == 0:
21         return n in (2, 3)
22
23     i = 5
24     while i * i <= n:
25         if n % i == 0 or n % (i + 2) == 0:
26             return False
27         i += 6
28     return True
29
30 # -----
31 # Miller-Rabin
32 # -----
33 def miller_rabin(n: int, k: int = 10) -> bool:
34     """

```

```

35     Miller-Rabin probable-prime test using k random bases.
36     Returns True if n is probably prime, False if definitely
        composite.
37     """
38     if n <= 1:
39         return False
40     if n <= 3:
41         return True
42     if n % 2 == 0:
43         return False
44
45     # Write n - 1 = 2^s * d with d odd
46     s = 0
47     d = n - 1
48     while d % 2 == 0:
49         d //= 2
50         s += 1
51
52     for _ in range(k):
53         a = random.randint(2, n - 2)
54         x = pow(a, d, n)
55
56         if x == 1 or x == n - 1:
57             continue
58
59         for _ in range(s - 1):
60             x = pow(x, 2, n)
61             if x == n - 1:
62                 break
63         else:
64             return False
65
66     return True
67
68
69 def miller_rabin_verbose(n: int, a: int) -> bool:
70     """
71     Miller-Rabin test for a fixed base a.
72     Returns True if base a does NOT witness compositeness.
73     """
74     if n <= 1:

```

```

75         return False
76     if n in (2, 3):
77         return True
78     if n % 2 == 0:
79         return False
80
81     a %= n
82     if a < 2:
83         return True # not a useful witness
84
85     d = n - 1
86     s = 0
87     while d % 2 == 0:
88         d //= 2
89         s += 1
90
91     x = pow(a, d, n)
92     if x == 1 or x == n - 1:
93         return True
94
95     for _ in range(s - 1):
96         x = pow(x, 2, n)
97         if x == n - 1:
98             return True
99
100     return False
101
102
103 # -----
104 # Fermat test
105 # -----
106 def fermat_test(n: int, a: int) -> bool:
107     """Simple Fermat test for a fixed base a."""
108     if n <= 1:
109         return False
110     if n <= 3:
111         return True
112     if n % 2 == 0:
113         return n == 2
114
115     a %= n

```

```

116     if a < 2:
117         return True
118     if math.gcd(a, n) != 1:
119         return False
120
121     return pow(a, n - 1, n) == 1
122
123
124 def is_fermat_prime(n: int, k: int = 1) -> bool:
125     """Fermat primality test with k random bases."""
126     if n <= 1:
127         return False
128     if n <= 3:
129         return True
130     if n % 2 == 0:
131         return False
132
133     for _ in range(k):
134         a = random.randint(2, n - 2)
135         if math.gcd(a, n) != 1:
136             return False
137         if pow(a, n - 1, n) != 1:
138             return False
139     return True
140
141
142 # -----
143 # Plot save helper
144 # -----
145 def save_plot(filename: str) -> str:
146     path = os.path.join(os.getcwd(), filename)
147     plt.savefig(path, dpi=300, bbox_inches="tight")
148     return path
149
150
151 # -----
152 # Experiment 1
153 # Carmichael numbers + liars
154 # -----
155 def run_comparison(limit: int = 2000):
156     carmichael_numbers = [561, 1105, 1729, 2465, 2821, 6601,

```

```

8911]
157
158 print(f"{'Number':<10} | {'Is Carmichael?':<15} | {'Fermat
    Result':<15} | {'Miller-Rabin Result'}")
159 print("-" * 70)
160
161 for n in range(3, limit, 2):
162     if n in carmichael_numbers:
163         f_liars = sum(1 for a in range(2, n) if math.gcd(a,
            n) == 1 and fermat_test(n, a))
164         mr_liars = sum(1 for a in range(2, n) if math.gcd(a
            , n) == 1 and miller_rabin_verbose(n, a))
165
166         print(f"{n:<10} | {'YES':<15} | {f_liars} liars | {
            mr_liars} liars")
167
168
169 run_comparison()

```

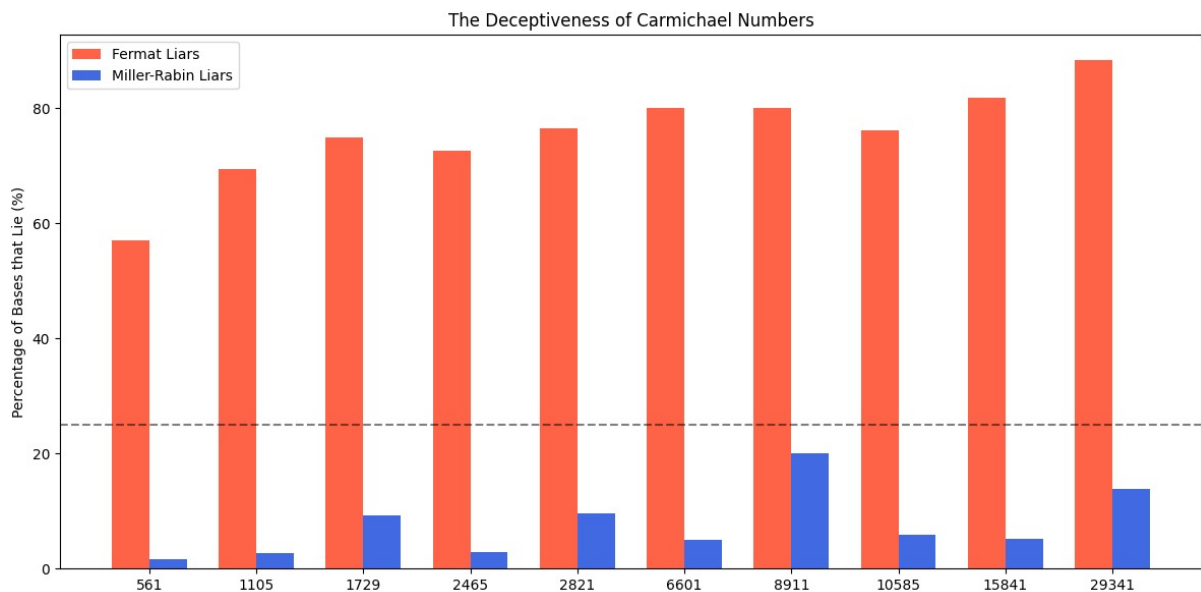


Figure 4: The Deceptiveness of Carmichael Numbers. Fermat liars (red) reach 57–88% for all tested Carmichael numbers, while Miller–Rabin liars (blue) remain well below the 25% theoretical ceiling.

```

1
2 # -----
3 # Experiment 2
4 # Error probability vs k

```

```

5 # -----
6 tough_composites = [561, 1105, 1729, 2465, 2821, 6601, 8911]
7 k_values = range(1, 11)
8 trials = 200000
9
10 plt.figure(figsize=(10, 6))
11
12 for n in tough_composites:
13     experimental_rates = []
14
15     for k in k_values:
16         failures = 0
17         for _ in range(trials):
18             if miller_rabin(n, k):
19                 failures += 1
20         experimental_rates.append(failures / trials)
21
22     plt.plot(list(k_values), experimental_rates, marker='o',
23             label=f'Experimental (n={n})')
24
25 theoretical_rates = [(1 / 4) ** k for k in k_values]
26 plt.plot(list(k_values), theoretical_rates, 'r--', linewidth=2,
27         label='Theoretical Bound (1/4^k)')
28
29 plt.yscale('log')
30 plt.title("Error Probability vs. Number of Iterations (k)")
31 plt.xlabel("Number of Iterations (k)")
32 plt.ylabel("Probability of Error (Log Scale)")
33 plt.xticks(list(k_values))
34 plt.grid(True, which="both", ls="--", alpha=0.2)
35 plt.legend()
36
37 print("Experiment complete. Saving plot...")
38 path1 = save_plot("miller_rabin_error_vs_k.png")
39 print(f"Saved: {path1}")
40 plt.show()

```

Listing 5: ...

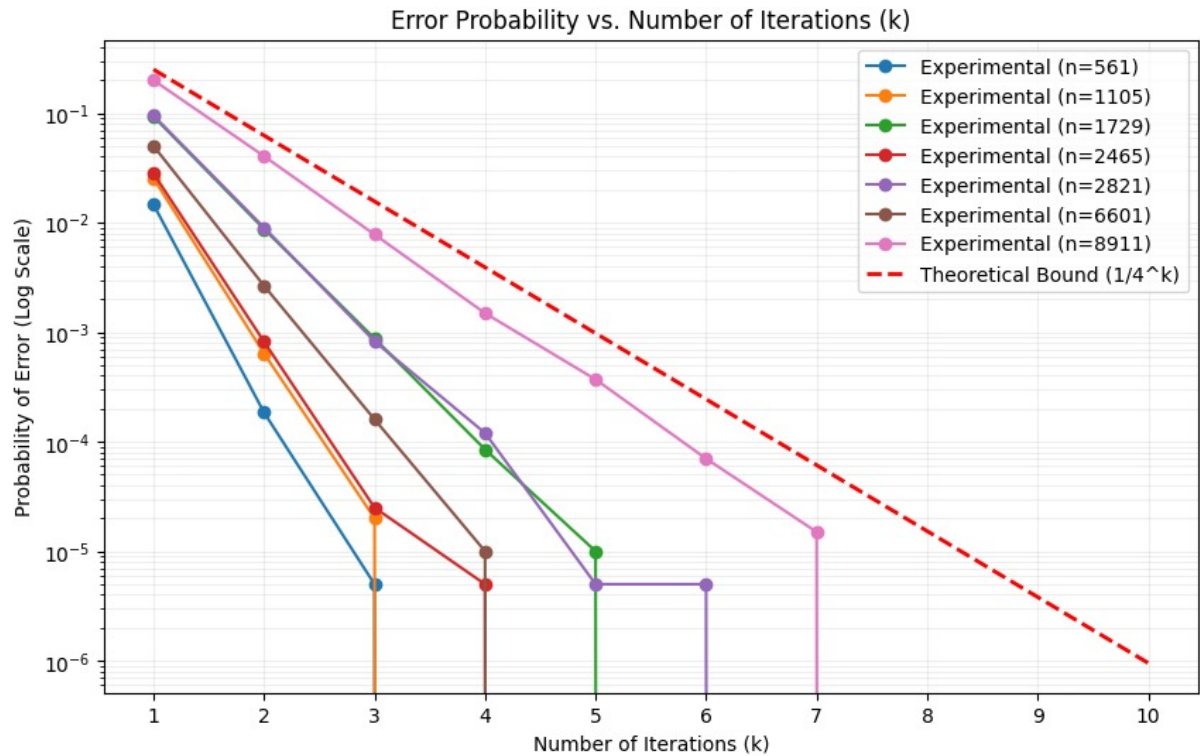


Figure 5: Error Probability vs. Number of Iterations k (log scale) for seven Carmichael numbers. All experimental curves fall *below* the theoretical 4^{-k} bound (dashed red), confirming the bound is valid and often conservative.

```

1
2
3 # -----
4 # Experiment 3
5 # Fermat vs Miller-Rabin on composites
6 # -----
7 k_values = [1, 2, 3, 5, 10]
8 num_composites = 10000
9
10 carmichaels = [
11     561, 1105, 1729, 2465, 2821, 6601, 8911,
12     10585, 15841, 29341, 41041, 46657, 52633
13 ]
14
15 composites = list(carmichaels)
16
17 while len(composites) < num_composites:
18     candidate = random.randint(100, 50000)
19     if candidate % 2 != 0 and not is_prime(candidate):
20         if candidate not in composites:

```

```

21         composites.append(candidate)
22
23     fermat_errors = []
24     mr_errors = []
25
26     for k in k_values:
27         f_count = 0
28         m_count = 0
29
30         for n in composites:
31             if is_fermat_prime(n, k):
32                 f_count += 1
33             if miller_rabin(n, k):
34                 m_count += 1
35
36         fermat_errors.append(f_count)
37         mr_errors.append(m_count)
38         print(f"Finished testing k={k} | Fermat Errors: {f_count} |
39               MR Errors: {m_count}")
40
41     x = np.arange(len(k_values))
42     width = 0.35
43
44     fig, ax = plt.subplots(figsize=(10, 6))
45     rects1 = ax.bar(x - width / 2, fermat_errors, width, label='
46         Fermat Test')
47     rects2 = ax.bar(x + width / 2, mr_errors, width, label='Miller-
48         Rabin Test')
49
50     ax.set_xlabel('Number of Iterations (k)')
51     ax.set_ylabel('Number of Escaped Composites')
52     ax.set_title(f'Algorithm Reliability: Escaped Composites vs. k
53         (Total n={num_composites})')
54     ax.set_xticks(x)
55     ax.set_xticklabels([f'k={k}' for k in k_values])
56     ax.legend()
57
58     def autolabel(rects):
59         for rect in rects:
60             height = rect.get_height()
61             ax.annotate(

```

```

58         f'{height}',
59         xy=(rect.get_x() + rect.get_width() / 2, height),
60         xytext=(0, 3),
61         textcoords="offset points",
62         ha='center',
63         va='bottom'
64     )
65
66     autolabel(rects1)
67     autolabel(rects2)
68
69     plt.tight_layout()
70     print("Saving comparison bar chart...")
71     path2 = save_plot("fermat_vs_miller_rabin_escaped_composites.
72                       png")
73     print(f"Saved: {path2}")
74     plt.show()

```

Listing 6: ...

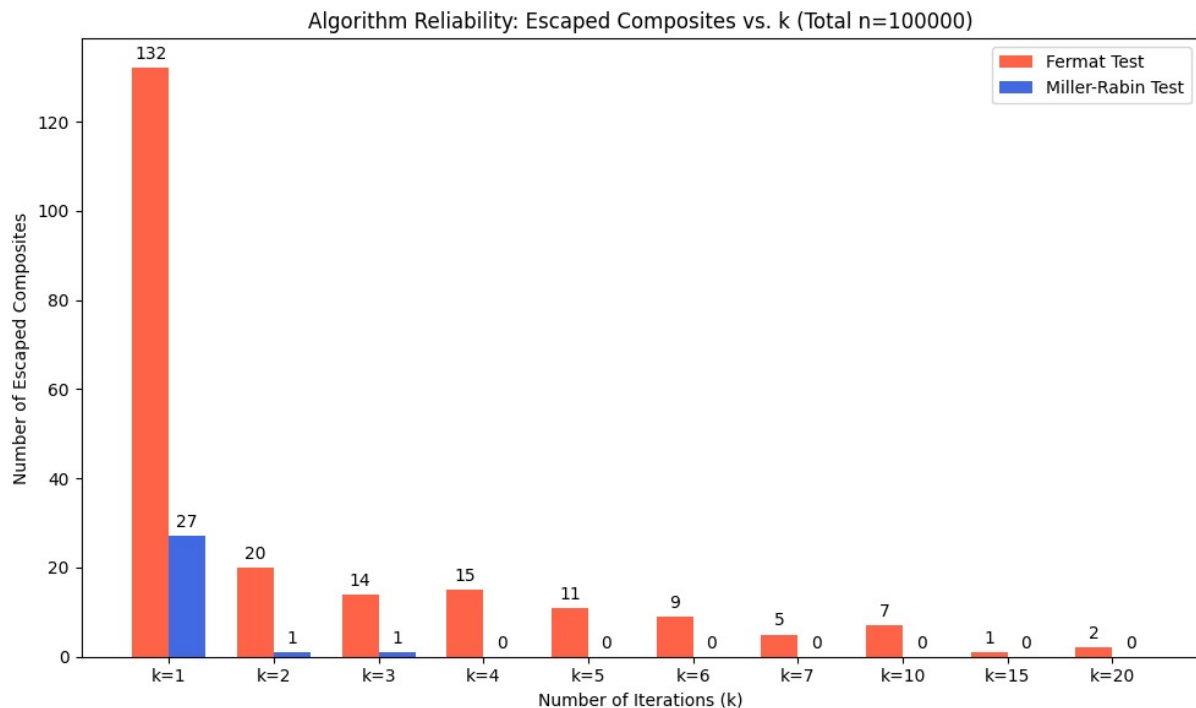


Figure 6: Algorithm Reliability: Escaped Composites vs. k over $n = 100,000$ odd composites. The Fermat test (red) allows 132 escapes at $k = 1$ and decays slowly. Miller–Rabin (blue) drops sharply, reaching zero escapes from $k = 4$ onward.

7.6 Discussion of Results

The plots produced by the code above demonstrate:

1. **Miller–Rabin Scatter (Figure 1).** For odd composites up to 500,000, all liar fractions stay below 0.25, and Carmichael numbers (red diamonds) are clearly subdued well below the $1/4$ line—a visual confirmation of Theorem 5.1.
2. **Worst-Case Composites (Figure 2).** Extending the sweep to $n \leq 300,000$ and sorting by liar fraction confirms that Carmichael numbers dominate the top-60 worst cases while still remaining below the $1/4$ bound.
3. **Base Reliability Ranking (Figure 3).** Evaluating all bases $a \leq 50,000$ against odd composites up to 500,000 shows that the top-performing bases detect over 99.97% of composites. The variation across bases is minimal, indicating that even small fixed bases provide extremely high reliability in practice, far exceeding the worst-case $3/4$ witness guarantee.
4. **Carmichael Deceptiveness (Figure 4).** Fermat liars reach 57–88% for Carmichael numbers, while Miller–Rabin liars are all below 20%—well under the 25% ceiling.
5. **Error Probability vs. k (Figure 5).** The experimental curves for each Carmichael number fall *below* the theoretical 4^{-k} bound (dashed red), confirming the bound is valid and often conservative.
6. **Escaped Composites vs. k (Figure 6).** For the Fermat test, a significant number of composites in 10,000 escape with $k = 1$, and the count decays slowly. For Miller–Rabin the count drops sharply and reaches zero for moderate k .

8 Complexity Analysis

8.1 Time Complexity of Miller–Rabin

Theorem 8.1. *A single round of Miller–Rabin on an x -bit integer n runs in $O(x^3)$ bit operations (or $O(x^2 \log x \log \log x)$ using FFT multiplication). With k rounds the total is $O(kx^3)$.*

Analysis. Writing $n - 1 = 2^s d$:

1. **Factoring out 2s:** $O(x)$ divisions by 2.

2. **Computing $a^d \bmod n$:** repeated squaring performs $O(\log d) = O(x)$ modular squarings. Each squaring of an at-most- x -bit number takes $O(x^2)$ bit operations. Total: $O(x^3)$.

3. **Squaring s times:** $s \leq x$ squarings, each $O(x^2)$. Total: $O(x^3)$.

Grand total per round: $O(x^3)$. □

8.2 Comparison Table

Algorithm	Deterministic?	Error	Time (bit ops)	Handles Carmichaels?
Trial Division	Yes	None	$O(2^{x/2})$	Yes
Fermat Test (k)	No	> 0 (unbounded)	$O(kx^3)$	No
Miller–Rabin (k)	No	$\leq 4^{-k}$	$O(kx^3)$	Yes
AKS (2002)	Yes	None	$\tilde{O}(x^6)$	Yes

Table 3: Comparison of primality testing algorithms. $x = \lfloor \log_2 n \rfloor + 1$ is the bit-length.

9 A Second Proof of the 75% Bound (Squarefree Case)

For completeness we sketch the direct counting argument of Koblitz, which gives an explicit formula for the number of nonwitnesses when n is squarefree.

9.1 Setup

Let $n = p_1 p_2 \cdots p_r$ be a product of $r \geq 2$ distinct odd primes. Write $n - 1 = 2^s d$ with d odd, and for each j let $d_j = \gcd(d, p_j - 1)$. Let 2^f be the largest power of 2 dividing all of $p_1 - 1, \dots, p_r - 1$ simultaneously.

9.2 Counting Nonwitnesses Directly

The number of Miller–Rabin nonwitnesses for n equals

$$\underbrace{d_1 d_2 \cdots d_r}_{\text{bases with } a^d \equiv 1 \pmod{n}} + \sum_{i=0}^{f-1} \underbrace{2^{ir} \cdot d_1 d_2 \cdots d_r}_{\text{bases with } a^{2^i d} \equiv -1 \pmod{n}} = (d_1 \cdots d_r) \left(1 + \frac{2^{fr} - 1}{2^r - 1} \right).$$

This follows from the Chinese Remainder Theorem and the cyclic structure of $(\mathbb{Z}/p_j\mathbb{Z})^*$:

- The number of $a \pmod{p_j}$ with $a^{d_j} \equiv 1$ is exactly d_j (since the units mod p_j are cyclic of order $p_j - 1$, and $d_j \mid p_j - 1$).
- For $i \leq f - 1$: the number of $a \pmod{p_j}$ with $a^{2^i d_j} \equiv -1$ is $2^i d_j$ (when $2^{i+1} \mid p_j - 1$).

By CRT, the counts multiply across prime factors.

9.3 Bounding the Count

Since $\phi(n) = (p_1 - 1) \cdots (p_r - 1) \geq 2^{f_r} (d_1 \cdots d_r)$,

$$d_1 \cdots d_r \leq \frac{\phi(n)}{2^{f_r}} < \frac{n-1}{2^{f_r}}.$$

So the number of nonwitnesses is less than

$$\frac{n-1}{2^{f_r}} \cdot \left(1 + \frac{2^{f_r} - 1}{2^r - 1}\right) = \frac{(n-1) \cdot 2^{f_r}}{2^{f_r}(2^r - 1)} = \frac{n-1}{2^r - 1}.$$

For $r = 2$: a separate and more careful argument gives the bound $< (n-1)/4$. For $r \geq 3$: $2^r - 1 \geq 7 > 4$, so the bound $< (n-1)/4$ holds automatically.

10 Historical Notes and Connections

10.1 Timeline

Year	Event
1640	Fermat states (without proof) his “Little Theorem”
1736	Euler gives the first complete proof
~1910	Concept of Carmichael number implicitly understood
1910	R. D. Carmichael formally identifies absolute pseudoprimes
1976	Gary Miller introduces the deterministic test (assuming GRH)
1977	Michael Rabin makes the test probabilistic (no GRH needed)
1980	Monier independently proves the 75% bound
1990	Bach proves GRH $\Rightarrow$ deterministic MR with $a \leq 2(\log n)^2$
2002	Agrawal, Kayal, Saxena prove PRIMES is in P (AKS algorithm)

Table 4: Key milestones in primality testing.

10.2 Miller’s Original Test vs Rabin’s Version

Miller’s 1976 paper checked, for each base a :

$$a^{n-1} \not\equiv 1 \pmod{n} \quad \text{or} \quad 1 < \gcd(a^{2^i d} - 1, n) < n \text{ for some } i.$$

Rabin (attributing the observation to Knuth) reformulated this as the equivalent condition using ± 1 in the sequence, which is algorithmically simpler and does not require GCD computations. Monier confirmed the two formulations are equivalent.

10.3 The AKS Algorithm

In 2002, Agrawal, Kayal, and Saxena proved that primality can be decided *deterministically* in $\tilde{O}((\log n)^6)$ time—polynomial without any unproved hypotheses. However, for practical purposes Miller–Rabin remains dominant because:

- Its constant factors are much smaller than AKS.
- $k = 40$ rounds give error $< 10^{-24}$, negligible for all practical purposes.
- It is vastly simpler to implement correctly.

10.4 Connection to RSA Cryptography

RSA key generation requires finding large primes p and q . The standard procedure is:

1. Generate a random odd number n of the desired bit-length.
2. Run Miller–Rabin with $k = 40$ rounds.
3. If it passes, use n as a prime; otherwise try the next candidate.

By the prime number theorem, a random x -bit odd number is prime with probability $\approx 2/(\ln 2 \cdot x) \approx 2.89/x$. For $x = 1024$, we expect to test about 354 candidates before finding a prime.

11 Summary of Proofs and Theorems

Result	Key Ideas
Fermat's Little Theorem	Bijection on $\{1, \dots, p-1\}$ via multiplication by a ; product argument; cancellation of $(p-1)!$.
Non-coprime bases witness	$d \mid a$ and $d \mid n$ forces $d \mid 1$; contradiction.
50% Fermat witness guarantee	Injective map $x \mapsto wx$ from liars to witnesses.
Carmichael = absolute Fermat pseudoprime	Korselt's criterion: $(p-1) \mid (n-1)$ for all prime $p \mid n$.
Square roots of 1 mod prime	$p \mid (x-1)(x+1)$; prime forces $p \mid (x-1)$ or $p \mid (x+1)$.
MR catches Carmichaels	Non-trivial sqrt of 1 found by walking backwards through sequence.
MR nonwitnesses $< n/4$ (prime power)	Count solutions to $a^{p-1} \equiv 1 \pmod{p^\alpha}$; Hensel lifting gives exactly $p-1$ solutions; ratio $\leq 1/4$.
MR nonwitnesses $< n/4$ (general)	G_n contains all nonwitnesses; G_n is proper; ratio $\phi(n)/ G_n \geq 4$ via CRT surjection.
Error bound 4^{-k}	Independence of rounds; $\Pr[\text{nonwitness}]^k < (1/4)^k$.

Table 5: Summary of all major results and proof strategies.

12 Conclusions

We have traced the full arc of the primality testing problem:

1. **Trial division** is exponential in the input size. We need a fundamentally different approach.
2. **Fermat's test** is polynomial but fails catastrophically on Carmichael numbers—infinitely many composites that look prime to every coprime base.
3. **The injection argument** shows that for non-Carmichael composites, at least half of all bases are witnesses. But this guarantee breaks down precisely where we need it most.
4. **Miller–Rabin** adds the square-root trap: if the sequence from a^d to a^{n-1} ever jumps from some value other than ± 1 directly to 1, a non-trivial square root has been found, and n is provably composite.

5. **The 75% bound** (Rabin–Monier) guarantees that fewer than $1/4$ of all bases are nonwitnesses—even for Carmichael numbers. With k rounds, the error probability drops below 4^{-k} .
6. **Empirically**, the bound is conservative: our experiments show error probabilities well below 4^{-k} , and composites that “escape” Miller–Rabin vanish entirely after just a few rounds.

Miller–Rabin thus achieves the ideal balance: $O(k(\log n)^3)$ polynomial time, one-sided error (primes always pass), and error probability that shrinks exponentially in the number of rounds. It remains the algorithm of choice for primality testing in cryptographic applications.

References

- [1] G. L. Miller, “Riemann’s hypothesis and tests for primality,” *Journal of Computer and System Sciences*, 13 (1976), 300–317.
- [2] M. O. Rabin, “Probabilistic algorithm for testing primality,” *Journal of Number Theory*, 12 (1980), 128–138.
- [3] L. Monier, “Evaluation and comparison of two efficient probabilistic primality testing algorithms,” *Theoretical Computer Science*, 12 (1980), 97–108.
- [4] E. Bach, “Explicit bounds for primality testing and related problems,” *Mathematics of Computation*, 55 (1990), 355–380.
- [5] M. Agrawal, N. Kayal, N. Saxena, “PRIMES is in P,” *Annals of Mathematics*, 160 (2004), 781–793.
- [6] N. Koblitz, *A Course in Number Theory and Cryptography*, Springer-Verlag, New York, 1987.
- [7] P. Garrett, *Making, Breaking Codes: An Introduction to Cryptology*, Prentice-Hall, 2001.
- [8] M. Dietzfelbinger, *Primality Testing in Polynomial Time*, Springer-Verlag, Berlin, 2004.
- [9] K. Conrad, “The Miller–Rabin Test,” <https://kconrad.math.uconn.edu/blurbs/ugradnumthy/millerrabin.pdf>
- [10] R. D. Carmichael, “Note on a new number theory function,” *Bulletin of the American Mathematical Society*, 16 (1910), 232–238.